



Speeduino Manual

This manual is compiled from the Speeduino documentation wiki:

<https://speeduino.com/wiki/index.php>

Sun Nov 12 2017

Contents

1	Introduction	1
1.1	Introduction	1
1.2	Compiling and Installing Firmware	1
1.2.1	Requirements	2
1.2.2	Downloading the firmware	2
1.2.3	Compiling the firmware	2
1.2.4	Installing	3
1.2.5	Verifying Firmware	4
1.2.6	Troubleshooting	4
1.3	Connecting to TunerStudio	5
1.3.1	Downloading Tuner Studio	5
1.3.2	Setting up your project	5
1.4	Creating a TunerStudio Project	7
1.4.1	Configuring TunerStudio Project Properties	7
1.4.2	Settings Tab	7
1.4.3	Can Devices Tab	10
2	Hardware	11
2.1	Hardware requirements	11
2.1.1	Arduino	11
2.1.2	Inputs	11
2.1.3	Outputs	15
2.1.4	Auxiliary IO	16
2.1.5	CanBus	16
2.1.6	Third Party Addon's	17
2.1.7	GPIO for Speeduino	17
2.2	V0.3	17
2.2.1	Overview	17
2.2.2	Board Features	17
2.2.3	Physical Layout	18
2.2.4	Proto area	18
2.2.5	Board Assembly	18
2.2.6	Board Configuration	19

2.2.7	Board revisions	20
2.3	V0.4	20
2.3.1	Overview	20
2.3.2	Board Features	21
2.3.3	Physical Layout	21
2.3.4	Board Assembly	21
2.3.5	Board Configuration	22
2.3.6	Board revisions	23
3	Supported decoders	24
3.1	Missing Tooth	24
3.1.1	Overview	24
3.1.2	Applications	24
3.1.3	Tuner Studio Configuration	25
3.1.4	Trigger Pattern	25
3.2	Missing Tooth (cam)	26
3.2.1	Overview	26
3.2.2	Applications	26
3.2.3	Tuner Studio Configuration	26
3.2.4	Trigger Pattern	27
3.3	Dual Wheel	27
3.3.1	Overview	27
3.4	Basic Distributor	27
3.4.1	Overview	27
3.4.2	Trigger Signal	28
3.5	GM 7X	28
3.5.1	Overview	28
3.6	4G63	28
3.6.1	Overview	28
3.6.2	Applications	28
3.6.3	Tuner Studio Configuration	29
3.6.4	Timing adjustment	29
3.6.5	Trigger Pattern	29
3.7	GM 24X	29
3.7.1	Overview	29
3.7.2	Trigger Signal	30
3.8	Jeep 2000	30
3.8.1	Overview	30
3.8.2	Trigger Signal	30
3.9	Audi 135	30
3.10	Honda D17	30
3.10.1	Overview	30

3.10.2 Applications	31
3.10.3 Tuner Studio Configuration	31
3.10.4 Timing adjustment	31
3.10.5 Trigger Pattern	31
3.11 Miata 99	31
3.12 Non-360	31
3.12.1 Non-360 Decoder	31
3.13 Nissan 360	31
3.13.1 Overview	31
3.13.2 Applications	32
3.13.3 Tuner Studio Configuration	32
3.13.4 Timing adjustment	32
3.13.5 Trigger Pattern	32
3.14 Subaru 6/7	32
4 Configuration	33
4.1 Engine Constants	33
4.1.1 Settings[edit]	34
4.2 Injector Characteristics	35
4.2.1 Overview	35
4.2.2 Settings	35
4.3 Trigger Setup	35
4.3.1 Overview	35
4.3.2 Trigger Settings	36
4.3.3 Finding tooth #1 and trigger angle	37
4.4 IAT Density	37
4.4.1 Overview	37
4.4.2 Settings	37
4.5 Acceleration Enrichment	37
4.5.1 Overview	37
4.5.2 Theory	37
4.6 Rev Limits	38
4.6.1 Overview	38
4.6.2 Settings	39
4.7 Flex Fuel	39
4.7.1 Overview	39
4.7.2 Hardware	39
4.7.3 Tuning	40
4.8 Spark Settings	41
4.8.1 Overview	41
4.8.2 Settings	41
4.9 Dwell	43

4.10 Overview	43
4.11 Settings	44
4.11.1 Voltage correction	45
4.12 Cranking	45
4.12.1 Overview	45
4.12.2 Settings	46
4.13 Warmup	47
4.14 Idle	48
4.14.1 Overview	48

Chapter 1

Introduction

1.1 Introduction

This manual covers the hardware (sensors, wiring etc), software configuration and tuning elements related to running a Speeduino unit. When beginning with Speeduino, particularly if it is your first time installing and configuring an engine management system, this manual will assist in understanding Speeduino's capabilities and how it should be installed, both in terms of hardware and software/firmware.

Whilst this document will assist in providing information related to Speeduino's configuration, it does not cover advanced engine tuning, fuel / ignition strategies etc. As with any changes to engine management, the possibility of damage to hardware is very real should a system be configured incorrectly.

Getting Started

In terms of starting out with Speeduino, it is generally recommended to first upload the firmware to your Arduino and get it connecting to the tuning software (Tuner Studio) before moving on to hardware assembly or wiring etc. Software setup and configuration on Speeduino can be completed without the need for any additional hardware to be present (Beyond the arduino itself) and this allows exploration of the software and options available before either an outlay of significant funds or a significant investment of time.

The following sections of this manual cover how to compile and upload the firmware, as well as creating a new project in Tuner Studio. It is strongly recommended to read these as a starting point.

About this manual

The contents of this manual are compiled from the Speeduino wiki at <https://speeduino.com/wiki/index.php>. As an open source project, this documentation is growing continually and this offline manual is updated nightly with any changes that are made. This also means that you may come across gaps in the documentation where little information is currently provided. Please do not hesitate to post on the forum if there is something missing that you need critically (or even not so critically).

Additionally, if you would like to contribute to the Speeduino documentation, we would love to hear from you! The preferred method to request wiki access is either via the forum (<https://speeduino.com/forum/viewforum.php?f=9>) or via Slack (<http://slack.speeduino.com:3000/>)

1.2 Compiling and Installing Firmware

With the goal of maximum simplicity in mind, the process of compiling and installing the firmware is reasonably straightforward.

1.2.1 Requirements

- A Windows, Mac or linux PC
- One of the following:
 - **The Arduino IDE**. Current minimum version required is 1.6.7, although a newer version is recommended.
 - **PlatformIO**. Can be downloaded from <http://platformio.org/platformio-ide>
- A copy of the latest Speeduino codebase. See below.
- A copy of **TunerStudio** to test that the firmware has uploaded successfully

1.2.2 Downloading the firmware

There are two methods for obtaining the Speeduino firmware:

1. Regular, stable code drops are produced, typically on the last day of each month. Details of the latest code drop are below
2. If you want the latest and greatest (And occasionally flakiest) code, the git repository can be cloned and updated. See <https://github.com/noisymime/speeduino>

Latest Stable Firmware

- **Date:** November 9th 2017
- **Details:** See <https://speeduino.com/forum/viewtopic.php?f=13&t=1499>
- **Download:** <https://speeduino.com/wiki/images/f/f4/Speeduino-Oct17.zip>
- **Notes:**
 - **Remember to always reload the latest ini file into your TunerStudio project when you load new firmware** (See below for details)
 - **Tuner Studio version 3.0.2 or newer is required from July 2016 onwards!**
 - There is a known bug with these newer versions of the Arduino IDE when running on Windows XP. See this post for a workaround: <http://speeduino.com/forum/viewtopic.php?f=13&t=555&p=7664&hilit=xp#p7665>

Older firmware releases

If required, older firmware releases and details can be found at [Firmware History](#)

1.2.3 Compiling the firmware

- Start the IDE, select *File > Open*, navigate to the location you downloaded Speeduino to and open the **speeduino.ino** file.
- Set the board type: *Tools > Board > Arduino Mega 2560* or *Mega ADK* (This is the only board currently supported)
- Click the **Verify** icon in the top left corner (Looks like a tick) 

At this point you should have a compiled firmware! If you experienced a problem during the compile, see the [Troubleshooting](#) section below.

This video walks through the whole process of installing the firmware on your Arduino from scratch:

Optional (But recommended)

There is an option available for changing the compiler optimization level, which can improve . By default, the IDE uses the -Os compile option, which focuses on producing small binaries. As the size of the Speeduino code is not an issue but speed is a consideration, changing this to -O3 produces better results (Approximately 20% faster, with a 40% larger sketch size) To do this, you need to edit the platform.txt file:

- Make sure the Arduino IDE isn't running
- Open the platform.txt file which is in the following locations:
 - On Windows: c:\Program Files\Arduino\hardware\arduino\avr
 - On Mac: /Applications/Arduino/Contents/Resources/Java/hardware/arduino/avr/
 - On Linux:
- On the following 3 entries, change the Os to be O3:
 - compiler.c.flags
 - compiler.c.elf.flags
 - compiler.cpp.flags
- Save the file and restart the Arduino IDE

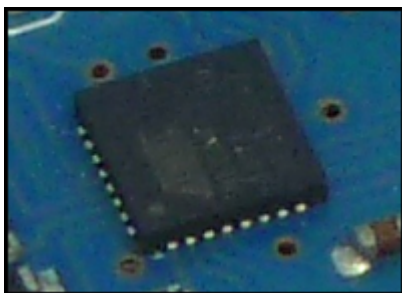
Note: This is NOT required if using PlatformIO, the above optimisation is applied automatically there

1.2.4 Installing

Once you've successfully compiled the firmware, installation on the board is trivial.

- Plug in your Mega 2560 to a free USB port
- If you're running **Windows** and this is the first time you've used an Arduino, you may need to install drivers for the Arduino serial chip. All official boards, and many non-official ones, use the ATmega16U2 or 8U2, whereas many of the cheaper 2560 based boards utilise the CH340G IC. The serial chips can generally be identified by appearance:

ATmega16U (square IC) - drivers included in Windows, MacOS and linux:



or

WCH CH340G (Rectangular IC) - drivers from **WCH**:



- Select the serial port to upload to: *Tools > Serial Port*
- Hit the Upload button from the top left corner (Looks like an arrow point to the right)

Assuming all goes well, you should see the IDE message that avrdude is done, similar to this:

1.2.5 Verifying Firmware

The firmware is now loaded onto your board and you are now able to move onto [Connecting to TunerStudio](#).

Optionally, you may perform a verification of the firmware by using the Arduino IDE's Serial Monitor. This can be started by selecting 'Serial Monitor' from the Tools menu.

In the window that appears, enter a capital "S" (no quotes) and press *Enter*. The Mega should respond with the year and month of the code version installed (xxxx.xx):

```
Speeduino 2017.03
```

NOTE: Ensure the baud rate is set to 115200

You can also enter "?" for a list of queries from your Mega:

```
=== Command Help ===
```

All commands are single character and are concatenated with their parameters without spaces. Some parameters are binary and cannot be entered through this prompt by conventional means.

Syntax: <command>+<parameter1>+<parameter2>+<parameterN>

```
=== List of Commands ===
```

```
A - Displays 31 bytes of currentStatus values in binary (live data)
B - Burn current map and configPage values to eeprom
C - Test COM port. Used by Tunerstudio to see whether an ECU is on a given serial
  port. Returns a binary number.
L - Displays map page (aka table) or configPage values. Use P to change page (not
  every page is a map)
N - Print new line.
P - Set current page. Syntax: P+<pageNumber>
R - Same as A command
S - Display signature number
Q - Same as S command
V - Display map or configPage values in binary
W - Set one byte in map or configPage. Expects binary parameters.
  Syntax: W+<offset>+<newbyte>
t - Set calibration values. Expects binary parameters. Table index is either 0,
  1, or 2. Syntax: t+<tbl_idx>+<newValue1>+<newValue2>+<newValueN>
Z - Display calibration values
T - Displays 256 tooth log entries in binary
r - Displays 256 tooth log entries
? - Displays this help page
```

You can now test your new Speeduino code by starting TunerStudio and attempting to connect. See [Connecting to TunerStudio](#) for more details

1.2.6 Troubleshooting

Incorrect Arduino board selected

If you see the following (or similar) errors when trying to compile the firmware and the solutions:

```
scheduler.ino:317:7: error: 'OCR4A' was not declared in this scope  
scheduler.ino:323:8: error: 'TIMSK5' was not declared in this scope  
scheduler.ino:323:25: error: 'OCIE4A' was not declared in this scope
```

You may have the wrong kind of Arduino board selected. Set the board type by selecting *Tools > Board > Arduino Mega 2560* or *Mega ADK*

Entire Speeduino project is not opened

The following can occur if you have only opened the speeduino.ino file rather than the whole project.

```
speeduino.ino:27:21: fatal error: globals.h: No such file or directory
```

Make sure all the files are contained within the same directory, then select *File->Open* and find the speeduino.ino file. If you have opened the project correctly, you should have multiple tabs along the top:

1.3 Connecting to TunerStudio

Tuner Studio is the recommended tuning interface for the Speeduino. It runs on Windows, Mac and linux and provides configuration, tuning and logging capabilities.

Once you have the firmware compiled and uploaded to your Arduino, you're ready to setup Tuner Studio in order to configure and monitor it. If you haven't yet compiled and uploaded the firmware, refer to the [Compiling and Installing Firmware](#) page.

1.3.1 Downloading Tuner Studio

If you haven't already, grab a copy of Tuner Studio from: (<http://www.tunerstudio.com/index.php/downloads>) Tuner Studio is available for Windows, Mac and linux and will run on most PCs as it's system requirements are fairly low.

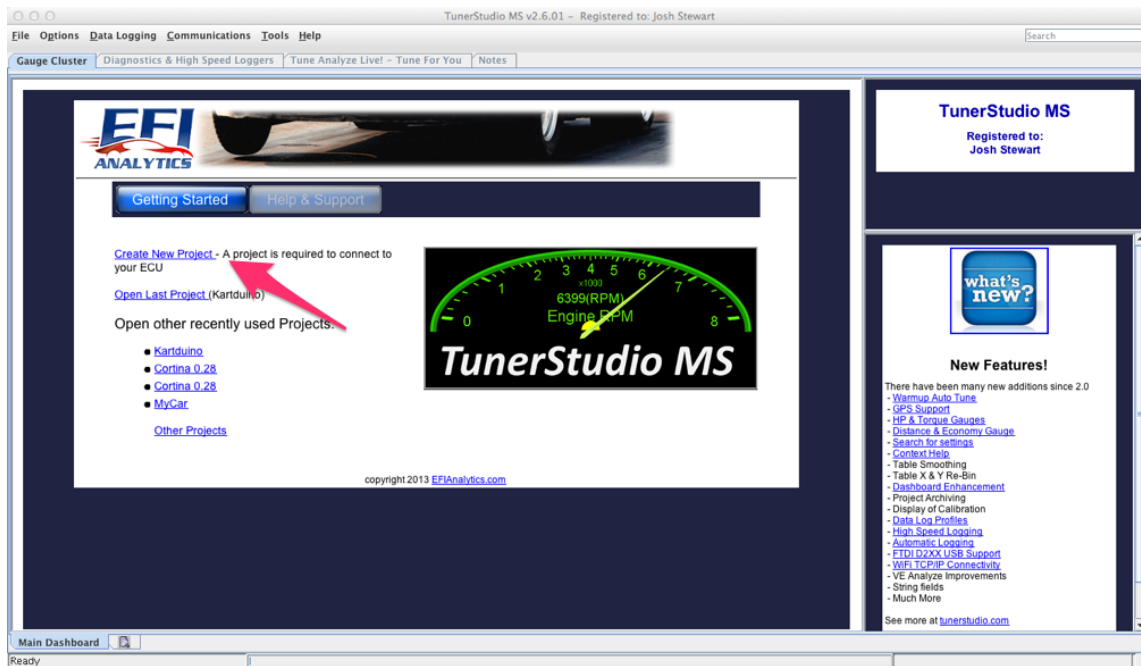
The current minimum version of TunerStudio required is 3.0.7, but the latest version is usually recommended.

If you find Tuner Studio to be useful, please consider paying for a license. This is a fantastic program from a single developer that rivals the best tuning software in the world, it's worth the money.

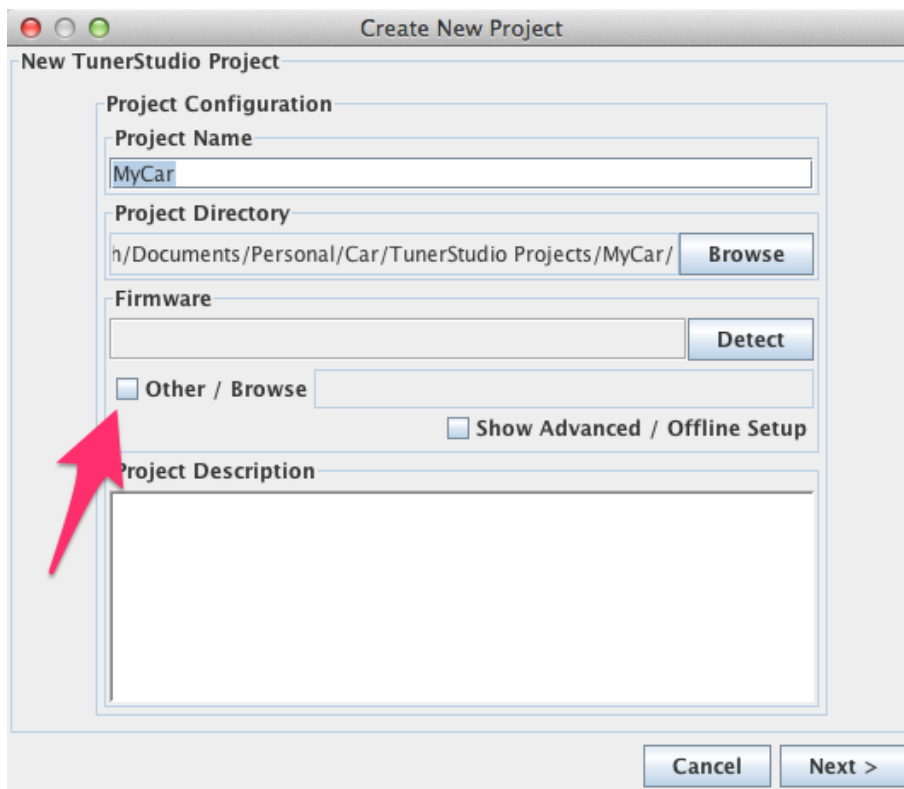
1.3.2 Setting up your project

Create new project

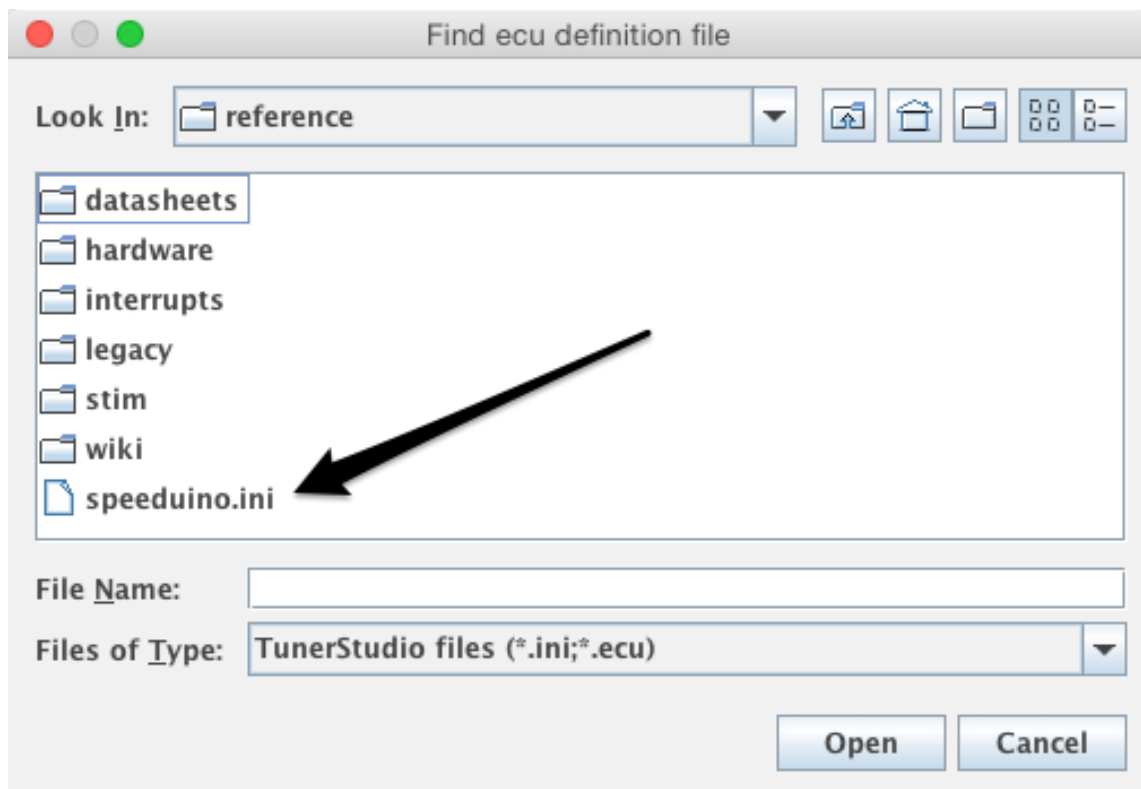
When you first start TunerStudio, you'll need to setup a new project which contains the settings, tune, logs etc. On the start up screen, select 'Create new project'



Give you project a name and select the directory you want the project to be stored in. Tuner Studio then requires a firmware definition file in order to communicate with the arduino. Tick the 'Other / Browse' button.



Then browse to the Speeduino source directory, enter the reference subfolder and select speeduino.ini file



Configuration options

Set the configuration parameters for your project. These can be changed any time later on, so don't worry if you don't have them at this time.

Comms settings

Select your comms options. The exact port name will depend on which operating system you are running and this will be the same as in the Arduino IDE. Baud rate should be 115200.

Load base tune

Once the project is created, you'll need to load in a base tune to ensure that all values are at least somewhat sane. Failure to do this can lead to very strange issues and values in your tune.

In the Speeduino reference directory, you will find the base tune file to be opened:

And that's it! Tuner Studio should now attempt to connect to the Arduino and show a realtime display of the ECU.

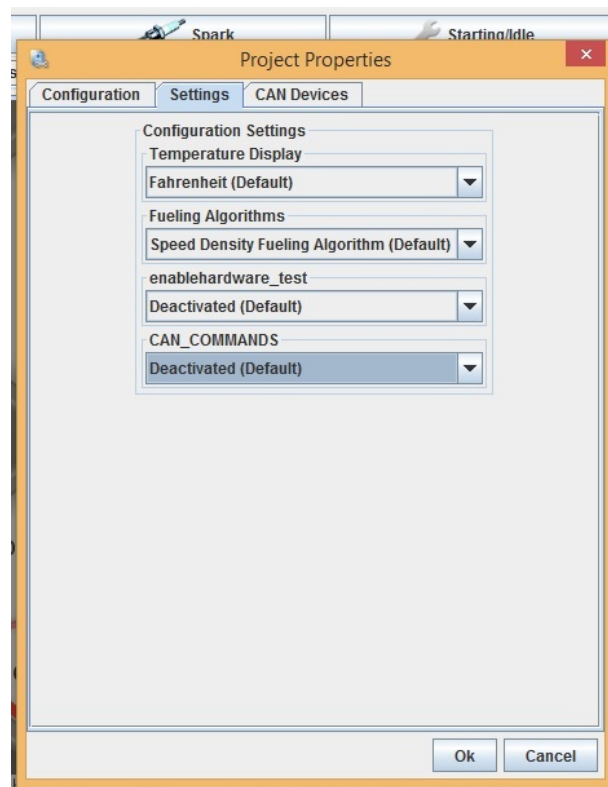
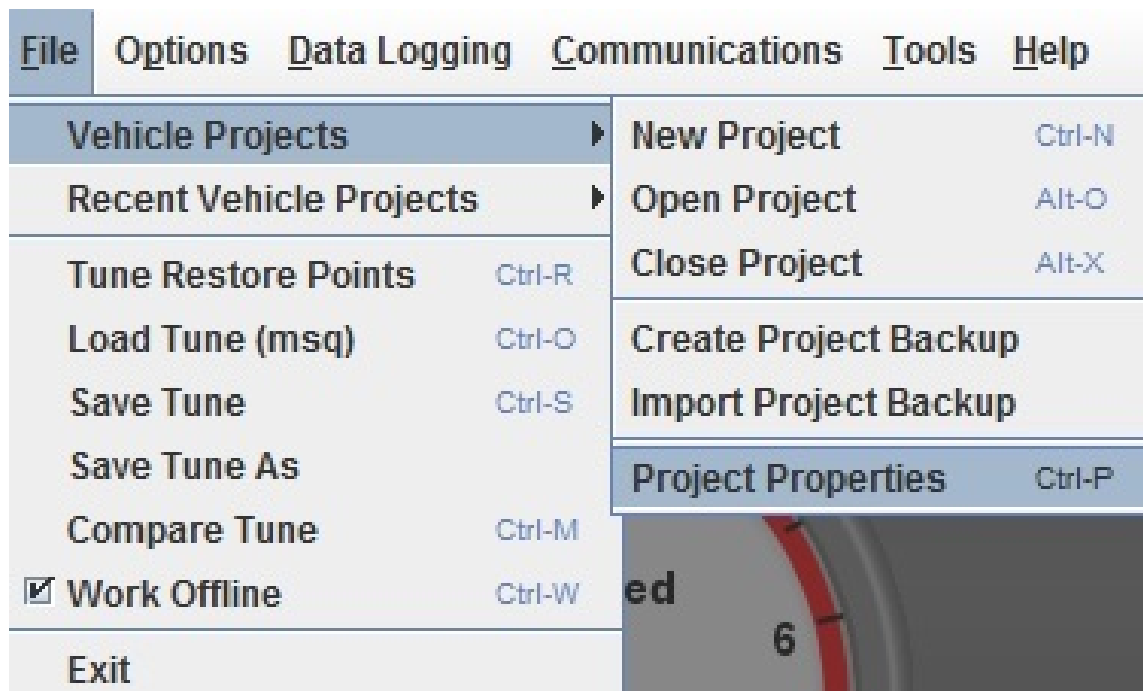
1.4 Creating a TunerStudio Project

1.4.1 Configuring TunerStudio Project Properties

The menu option for the project properties page can be found here

Once opened this page will be seen.

1.4.2 Settings Tab



Temperature Display

Options are :

- Fahrenheit(Default)
- Celsius

Fueling Algorithms

Options are :

- Speed Density Fueling Algorithm (default)
- Alpha-N Fueling Algorithm

Enable_hardware_test

Default option is disabled. If Enabled an additional Tab will appear on the tuning page



Clicking on this will open further options



- Output Testing

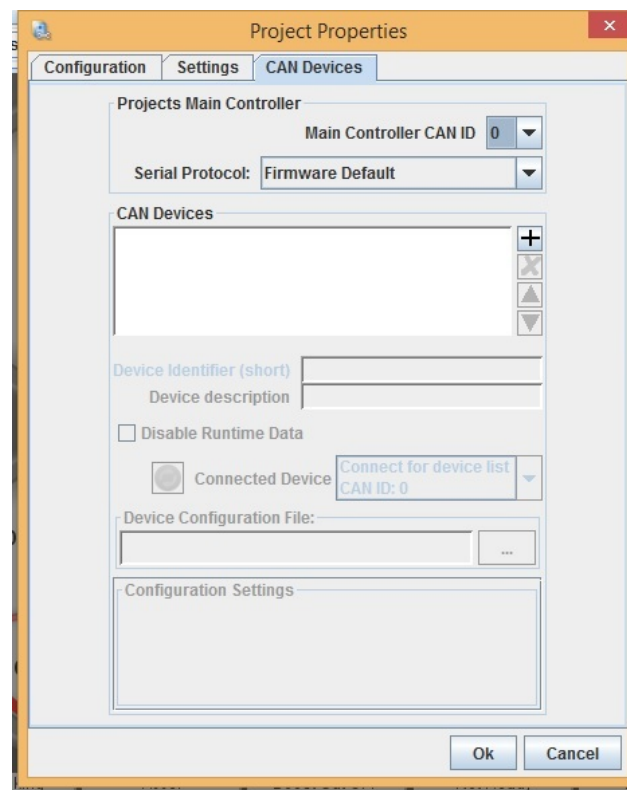
Hardware test page.

- Input Testing

CAN_COMMANDS

Default option is disabled

1.4.3 Can Devices Tab



Chapter 2

Hardware

2.1 Hardware requirements

2.1.1 Arduino

Speeduino uses the Arduino Mega 2560 R3 as the controller. All official and most clone boards appear to work fine. Note that there are minor differences with the Mega R2 board that can cause some small issues, however as it is nearly impossible to find an R2 board currently (R3 has been the available version for a long time) this should not be an issue for most people.

*Update: While the connection issues with some clone boards appear to have been resolved, an alternative solution is to use Bluetooth for wireless tuning of any Mega version, as the **Bluetooth** option bypasses the on-board UART (serial) chip.*

2.1.2 Inputs

Crank sensor

This is arguably the most important sensor for Speeduino to function correctly. The signal going to the Arduino must be a 0v-5v square wave series of pulses representing teeth on a wheel running at crank (or cam) speed. Many Hall and 'opto' sensors meet this spec. If only a crankshaft trigger wheel is used (no cam signal), the crank wheel must have a 'missing' tooth in order to provide position information as well as the engine RPM. Tested missing-tooth wheels currently are 4-1, 12-1, 36-1 and 60-2.

Alternatively (and necessary for full-sequential injection) an added cam signal with or without crank wheel missing teeth. These setups are indicated by the added "/x", such as 60-2/1, for a 60-tooth crank wheel, with 2 missing teeth, and a 1-tooth cam signal per cycle.

VR (variable reluctance) sensors can also be used, however as the board does not contain any sort of signal conditioner, an additional module will be required. An 8-pin DIP socket is located on v0.3.x and v0.4.x series official boards for this purpose as IC3. The MAX9926 chip has been tested to work with virtually any type of input, and is available from the [Speeduino Store](#), however any similar module that outputs a 0v-5v square wave (LM1815, LM358 modules, etc.) should also work fine with VR sensor signals.

TPS

TPS sensor must be of the 3 wire potentiometer type, rather than the 2 wire on/off switches found on some throttles. If your TPS is a 3 wire sensor then it will almost certainly work, but remember to calibrate it within Tuner Studio when you first plug it in.

MAP (Manifold Pressure)

Recommended MAP sensor is the MPX4250 from Freescale, however many MAP sensors are supported. If you want to use one that is not included in the list (Under Tools->Map Calibration in TunerStudio) then please make a new thread in the forum requesting this. Other sensors can and will work just fine, but you will need to calibrate these within TunerStudio against a different set of values.

Temperature Sensors (CLT and IAT)

Any standard 2-wire thermistor sensor can be used for these temperature functions. Calibration of these sensors can be performed through the Tools menu in TunerStudio. The default bias resistor value is 2490 Ohms.

Exhaust Gas Oxygen Sensors (O2 and WBO2)

The type of O2 sensor (narrow or wide-band) must be selected in TunerStudio under *Tools > Calibrate AFR Table*.

Narrow-band NBO2 sensor signals are read directly by Speeduino. TunerStudio applies the standard non-linear 0-volt to 1-volt values for all standard NBO2 sensors automatically under calibration. Once set in calibration, Speeduino will use the designated NBO2 to adjust fueling according to the entries you make in the AFR table (*Tuning > AFR Table*), and the sensor is selected for type and parameters (or disabled) under *Tuning > AFR/O2*. Note that narrow-band sensors were designed to target stoichiometric AFR (Lambda 1.0) for efficient catalytic emissions control, and are generally not sufficiently accurate or suitable for efficient economy or power fueling.

Wide-band Speeduino cannot use WBO2 sensors directly, requiring an external controller to process the signal and to apply sensor heating control. Enter the controller brand and model from the list displayed. If the controller signal is generic linear or custom, select and enter the required information, or an option to install a custom INC file is available in the menu list.

Once set in calibration, Speeduino will use the designated WBO2 to adjust fueling according to the entries you make in the AFR table (*Tuning > AFR Table*), and the sensor is selected for type and parameters (or disabled) under *Tuning > AFR/O2*.

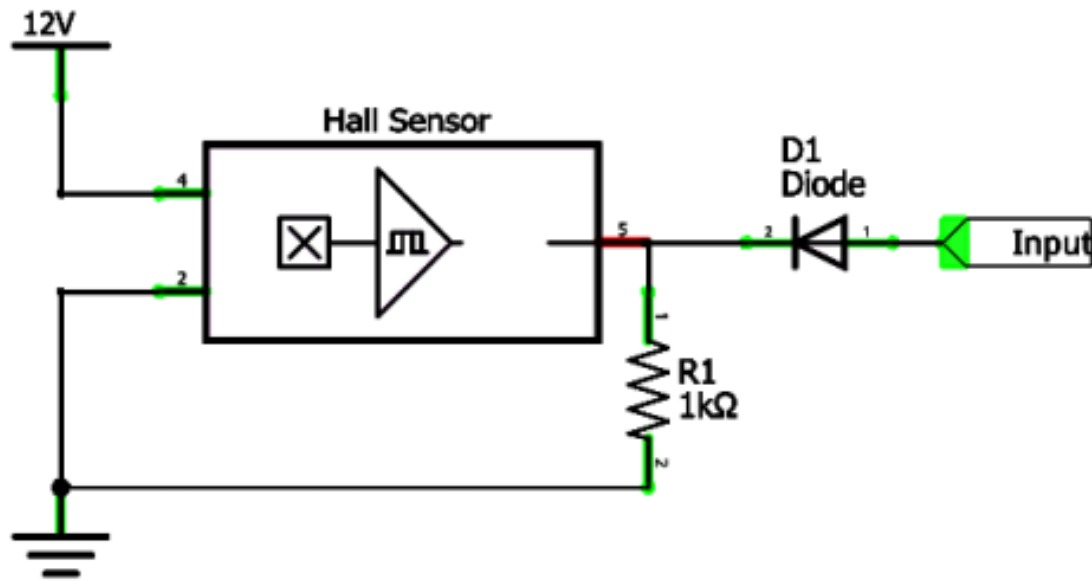
Application-Specific Inputs

Circuits and techniques Speeduino users have found useful for adapting or implementing certain inputs or functions.

Flex Fuel Sensor «placeholder»

Oil Pressure «placeholder»

12V Input Signal Some position sensors output a 12v signal. To correct this, and avoid damaging the Arduino, a circuit like the one in the diagram can be constructed. The resistor R1 is not always required, but will make sure that any output that is not high is pulled low. Along with this circuit use the pull-up jumper on the Speeduino. This will effectively change a 0v/12v into a 0v/5v signal.



Many thanks to PSIG for the info and diagram.

GM 7 / 8 pin Distributor Module The GM 7 / 8 pin modules have been used in a wide variety of GM engines from 4 cylinder to V8s (small and big block). The 8 pin distributor was also widely used in marine applications by Indmar, Mercruiser, and others.



Pin Module

GM7-



GM 8-Pin Ignition Module

ule

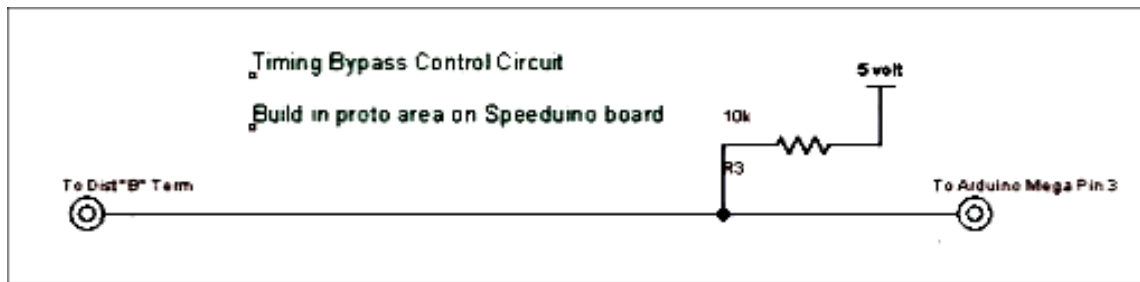
These modules are functionally equivalent. The 7 pin is used in the large coil-in-cap distributors while the 8 pin is used in the small cap distributors with remote mounted coils. The 8 pin has one additional terminal that provides a sensor ground. Both modules provide coil ground via the metal grommets used to secure them to the distributor.

These modules provide an excellent means for computer controlled timing while retaining the distributor. They were designed to be used with throttle body injection and port injection motors. They provide automatic coil current limiting (7.5 amps was the GM specification) and automatic dwell control. They can be adapted to other distributor applications that use either variable reluctor or hall type sensors.

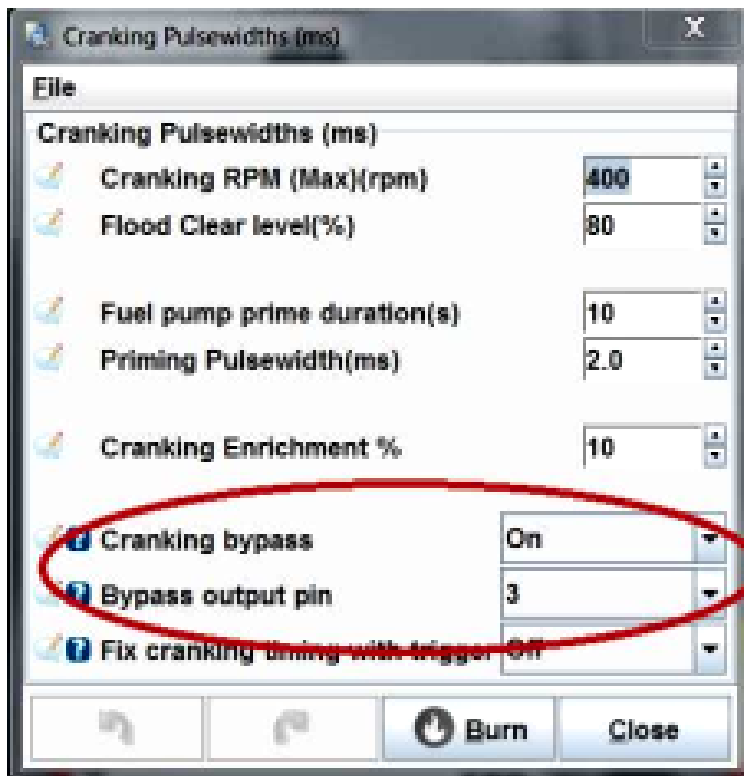
Pin Descriptions and connections • **"+"** : Battery voltage from a switched ignition source. Provides the power to operate the module.

- **"C-"** : Ignition coil negative connection.
- **"P & N"** : Positive and Negative of the distributor reluctor. Polarity is important. GM distributor connectors can only be connected one way. For use with other distributors, verify polarity of the reluctor leads.
- **"B"** : Ignition bypass. When cranking, grounding this line bypasses computer control of timing. The timing is controlled by the module only. This can be done using the Speeduino Cranking Bypass pin function (see below).
- **"R"** : Reference or tach signal. This outputs a 5 volt square wave that serves as the RPM1 input for the Speeduino. To use this, connect it to RPM1 and set JP2 to Hall and JP4 connected (ie, 5 volt pull up).
- **"E"** : Timing control signal. When pin B has 5 volts on it, the module allows Speeduino to control the timing using this pin. The output of Ign1 should be connected to this pin.
- **"G"** : (8 pin only) Signal ground. Should be connected to the Speeduino sensor ground. (Module ground is provided through the metal mounting grommets)

A timing bypass circuit must be constructed to utilize the Speeduino ignition timing control. The small circuit below should be built in the proto area.



In addition, the cranking bypass should be turned on and the bypass pin should be set to pin 3 in the Cranking Settings dialog (under Starting / Idle in TunerStudio):



Trigger settings (under Starting / Trigger Setup in TunerStudio) should be as shown below. You will need to adjust the trigger angle to get correct timing. Instructions for this are in the wiki.

Timing control is set in the Spark / Spark Settings dialog:

The module works well, however, some applications result in a noisy trigger signal. This shows as an unstable RPM (either at idle, or more frequently, at higher speeds). Trigger filtering may help, but a modification may be necessary to clean up the signal. The circuit below effectively cleans/filters the signal, allowing use with no trigger filtering by the Speeduino. It has been tested with single and 8-pole reluctors and modules from GM, Transpo and a no-name generic.

Many thanks to apollard for this outline information.

2.1.3 Outputs

Injectors

Speeduino injector drivers use on/off (not PWM) control and are designed to work with "High-Z" injectors. This type of injectors are also known as "saturated" or "high-impedance" that use full battery voltage to control the injector open cycle, and generally the impedance is greater than about 8 Ohms. If you are running "Low-Z" ("peak and hold" or PWM-controlled) injectors that are lower impedance, you will need to install series resistors on these to avoid damaging the board with excessive current. The resistor ohms and watt rating can be calculated by Ohm's Law, or

use an Internet calculator page such as the [Speeduino Injector Resistor Calculator](#).

Speeduino can drive up to 2 High-Z injectors per output channel.

Coils

Current versions of the Speeduino use low-power output signals, designed to work with external small-signal ignition coil drivers, whether a separate type (module or ICM, igniter, IGBT, etc.), or built into the coil assembly ('smart' coils). This method permits Speeduino to have great flexibility to control most types of ignition systems. Attaching the Speeduino outputs directly to a traditional high current ("dumb" or 2-pin) ignition coil without an ignition coil driver WILL cause damage to your Arduino.

How Speeduino controls ignition circuit power In prior history, the coil driver was a set of mechanical contact points ("points"), simply replaced today by an electronic version. The added coil driver can be anywhere from inside the Speeduino to inside the coil assembly; though near or in the coil reduces electrical noise:

The wasted-spark version is below to show how it is identical in operation, but with the high-voltage spark returning through the second spark plug to complete the circuit:

A good run down of 'smart' coil types can be found at: <http://www.megamanual.com/seq/coils.htm>. There are many ignition modules available that Speeduino can use to control standard coils, or for smart coils you can generally use 4 or 5-pin types as these will always be logic level, although some 3-pin coils are also of this variety. GM LS1/2 coils are an example of powerful smart coils that are commonly used and can usually be obtained easily and cheaply.

(Note: In the past, some ignition control modules with current limiting or dwell control features (e.g., 1970s GM HEI, Bosch '024' types, and Ford DSI) were referred-to as "smart" modules. While still true, common terminology of individual ignition coils with at least a driver integrated, or newer technology with greater controls (e.g., controlled spark duration or multi-spark) are all considered "smart" coils. You must know the control requirements of the specific drivers, control modules, or coils you intend to use in order to operate them properly with Speeduino.)

Aux Outputs

Low Current Some Speeduino versions include an 8-channel ULN2803A Darlington transistor array IC that is capable of switching up to 1/2 amp per channel. These auxiliary outputs are sufficient to switch small devices directly, or to switch power-handling devices, such as power MOSFETs and solid-state or electro-mechanical automotive relays. Configuration and settings of these outputs is described in the [Configuration / Tuning](#) section.

Medium Current v0.3x and later boards include medium-power MOSFET auxiliary outputs to switch up to 3 amps directly. These outputs are commonly used to operate idle valves, boost-control valves, VVT solenoids, etc., or to control relays for handling much larger loads, such as electric radiator fans. Configuration and settings of these outputs is described in the [Configuration / Tuning](#) section.

2.1.4 Auxiliary IO

On Arduino Mega2560 based Speeduino boards (official or just running current firmware) git commit 13f80e7 support is available for the external connection of 8 16bit analog inputs via [Serial3](#)

2.1.5 CanBus

As the Arduino mega2560 has no CanBus interface a separate "co-processor" interface has been designed. More information about this unit can be found here <https://github.com/Autohome2/Speeduino-can-interface>. This uses the functionality provided by the Serial3 port and connects via that port.

On the upcoming Teensy3.5 variant of Speeduino the CanBus code will be incorporated into the main system firmware as the Teensy3.5 has integrated CanBus and only requires a transceiver module added.

2.1.6 Third Party Addon's

In This section you will find information about third party designed hardware designed to be used in conjunction with the Speeduino ECU

2.1.7 GPIO for Speeduino

There are several variants of the GPIO , The firmware can be downloaded here <https://github.com/Autohome2/Speeduino-GPIO>

More information [GPIO_for_Speeduino](#)

2.2 V0.3

2.2.1 Overview

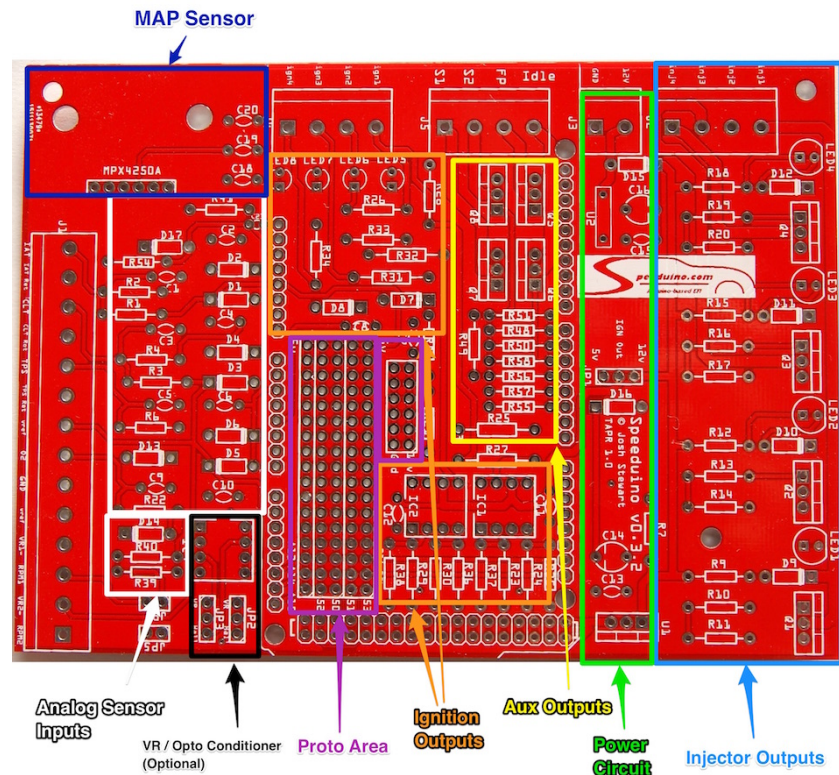
The v0.3 board is the first widely available Speeduino shield and is suitable for many typical 1-4 cylinder injection and ignition applications (Excluding direct injected engines).

2.2.2 Board Features

The v0.3 boards includes the following features:

- 4 injector channels
- 4 Ignition outputs
- Fully protected input channels for CLT, IAT, TPS and O2
- Optional VR conditioner mount on crank and cam inputs
- MAP sensor mount location
- 4 medium current spare outputs (Eg Fuel pump, thermo fan etc)
- All I/O through screw terminals on the board
- Proto area with i2c, SPI and power breakouts.

2.2.3 Physical Layout



2.2.4 Proto area

The proto area can be used for adding your own circuits on to Speeduino if required or simply as a convenient access point to various connections. The connections broken out to the proto board are:

- 5v and 12v
- Grounds
- SPI pins (MOSI, MISO, SCK and SS). Alternatively these can be used as generic digital IO (Arduino pins 50-53)
- I2C pins (SDA and SCL).
- 3 generic analog inputs (13-15)

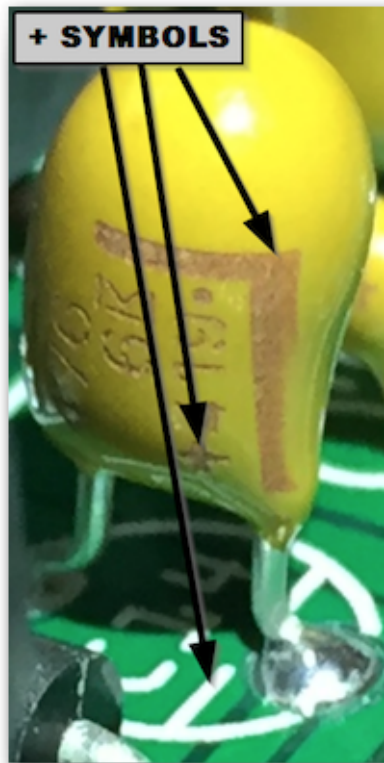
2.2.5 Board Assembly

Refer to the [Board revisions](#) for a link to the Bill of Materials (BOM) of your specific board.

Assembly of a complete board is relatively straightforward with all components being through hole and labelled on the board (See above mentioned BoM for parts list). Whilst it does not matter which order components are installed, the following is recommended for simplicity:

1. All resistors
2. All diodes (Including LEDS)
3. All capacitors

- (a) Take note that C14 and C16 are polarised capacitors, meaning that they must be put in the correct way around. The capacitors should be marked with a + sign on one side. On the PCB, the positive side is indicated by a line on the capacitor symbol.



1. All jumper headers (JP*)
2. Arduino pins:
 - (a) Suggested method: Break header pins into required lengths and insert into an Arduino Mega. Place the board over the top of the pins and solder in place
 - (b) Note that not all the pins on the end double row need to be populated (Though there's no harm in doing so). The odd numbered pins (Eg D23, D25 .. DD53) do not need pins on them.
3. IC sockets
4. MAP sensor (If used)
 - (a) **NOTE:** ALL self assembly boards have the MAP sensor with the hole at the top.
5. All screw terminals
6. All MOSFETs
7. Power regulator

Assembly Instruction video

2.2.6 Board Configuration

The board can be configured in multiple ways depending on the hardware you use and way your setup is configured.

Board default outputs

Multiple functions within Speeduino have adjustable outputs or can be set to Board Default. The following are the Default pin outs for the v0.3

Note: *These defaults are applicable to the Jan 2017 firmware and newer*

Optional Components

If using a VR crank sensor, the board will require the addition of a VR conditioner. The board has been designed to work with the dual VR conditioner from JBPerf (http://www.jbperf.com/dual_VR/index.html) which will plug directly in. Other VR conditioners will also likely work, but have not been tested. There is now also an official VR board that can be used, see link on the left.

SP721 Over-voltage Protection For users having difficulty obtaining the SP721 used in some versions, see info on the [SP721 Diode Alternate](#) page.

Jumper Configuration

Depending on the type of crank and cam sensors you have, there are a number of jumpers that will need to be set. The jumpers that need setting are:

- JP1 - This sets whether the Ignition outputs are 12v or 5v. Note that even if you set this to 12v you should ****NOT**** connect these directly to a high current coil. These outputs should only ever go to a logic level coil or an igniter
- JP2 - Whether or not the RPM1 (Crank) input should be routed via the (Optional) VR conditioner. This should be set to VR when using either a VR sensor or a hall sensor that switches between 0v-12v
- JP3 - Same as JP2, but for the RPM2 (Cam) input
- JP4 - 10k pullup resistor for RPM1 input. Should be jumpered ('On') when a sensor is used that switches between ground and floating (Which is most hall effect sensors)
- JP5 - Same as JP4, but for the RPM2 (Cam) input

To make this simpler, the most common sensor types and their required configurations are below:

2.2.7 Board revisions

2.3 V0.4

2.3.1 Overview

The v0.4 board is a testing board that was developed with the goals of reproducing the existing v0.3 boards capabilities, but with the following improvements:

- Lower cost (Primarily due to reduced size, but also some component changes)
- More compatible with off the shelf cases/enclosures
- Stepper-style IAC driver option
- Has a single 40 pin connector for all IO (Excluding 12v power)

Note that the v0.4 is **NOT** intended as a replacement for the v0.3 line of boards! The 2 are designed with different goals in mind. The v0.4 is intended to be integrated more closely into existing wiring, with the aim being that interface boards can be used to easily connect through the IDC40 connector. Unless you understand the interface on the v0.4 board and believe it is the best option for your install, the v0.3 may well be a better option for you.

2.3.2 Board Features

The v0.4 boards includes the following features:

- 4 injector channels
- 4 Ignition outputs
- Fully protected input channels for CLT, IAT, TPS and O2
- Optional VR conditioner mount on crank and cam inputs
- MAP sensor mount location
- DRV8825 stepper module mount location
- 4 medium-current spare outputs (e.g., fuel pump, thermo fan, boost control, VVT, etc)
- 5 unpopulated/configured optional low-current spare outputs in “proto” section, including tachometer-out
- A single 40-pin IDC connector includes all pins required for the board with the exception of the 12v input

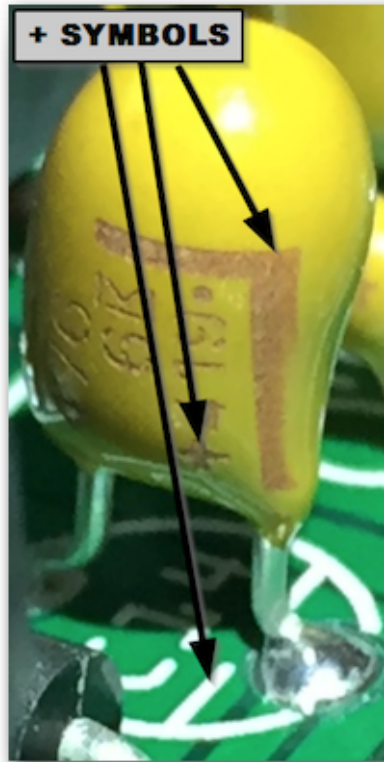
2.3.3 Physical Layout

Note that there are some differences between the various versions of the board, however the pinouts on the main IDC40 connector remain the same.

2.3.4 Board Assembly

Assembly of a complete board is virtually identical to the v0.3 and remains relatively straightforward with all components being through hole and labelled on the board. Whilst it does not technically matter which order components are installed, the following is recommended for simplicity:

1. All resistors
2. All diodes (Including LEDS)
3. All capacitors
 - (a) Take note that C14 and C16 are polarised capacitors, meaning that they must be put in the correct way around. The capacitors should be marked with a + sign on one side. On the PCB, the positive side is indicated by a line on the capacitor symbol.



1. All jumper headers (JP*)
2. Arduino pins:
 - (a) Suggested method: Break header pins into required lengths and insert into an Arduino Mega. Place the board over the top of the pins and solder in place
 - (b) Note that not all the pins on the end double row need to be populated (Though there's no harm in doing so). The odd numbered pins (Eg D23, D25 .. DD53) do not need pins on them.
3. IDC 40 connector
4. IC sockets
5. All screw terminals
6. All MOSFETs
7. Power regulator
8. MAP sensor (If used)
 - (a) **NOTE:** ALL self assembly boards have the MAP sensor with the hole at the top.

Assembly Instruction video

This video is for the v0.3 board, but it largely applies to v0.4 designs as well.

2.3.5 Board Configuration

The board can be configured in multiple ways depending on the hardware you use and way your setup is configured.

Board default outputs

Multiple functions within Speeduino have adjustable outputs or can be set to Board Default. The following are the Default pin outs for the v0.4

Optional Components

If using a VR crank sensor, the board will require the addition of a VR conditioner. The board has been designed to work with the dual VR conditioner from JBPerf (http://www.jbperf.com/dual_VR/index.html) which will plug directly in. Other VR conditioners will also likely work, but have not been tested. There is now also an official VR board that can be used, see link on the left.

SP721 Over-voltage Protection For users having difficulty obtaining the SP721 used in some versions, see info on the [SP721 Diode Alternate](#) page.

Jumper Configs

Depending on the type of crank and cam sensors you have, there are a number of jumpers that will need to be set. The jumpers that need setting are:

- JP1 - This sets whether the Ignition outputs are 12v or 5v. Note that even if you set this to 12v you should ****NOT**** connect these directly to a high current coil. These outputs should only ever go to a logic level coil or an igniter
- JP2 - Whether or not the RPM1 (Crank) input should be routed via the (Optional) VR conditioner. This should be set to VR when using either a VR sensor or a hall sensor that switches between 0v-12v
- JP3 - Same as JP2, but for the RPM2 (Cam) input
- JP4 - 1k pullup resistor for RPM1 input. Should be jumpered ('On') when a sensor is used that switches between ground and floating (Which is most hall effect sensors)
- JP5 - Same as JP4, but for the RPM2 (Cam) input

To make this simpler, the most common sensor types and their required configurations are below:

40-pin connection

You can solder wires directly to the board or use IDC (Insulation Displacement Contact) connectors. The 40-pin IDC is the connector that was used on computer drive ribbon cables for years and old computer cables can be used. A heavier cable, called DuPont cable is recommend for long term use though. Later in the IDE/ATA interfaces life the speed was increased and this required a new fine 80-wire cable. These cables are **NOT** compatible. Some of the pins are connected together causing the magic blue smoke to be released.

2.3.6 Board revisions

Chapter 3

Supported decoders

3.1 Missing Tooth

3.1.1 Overview

A missing tooth crank trigger is used as standard equipment by a number of OEMs, most notably Ford, but is also very popular as an aftermarket fitment.

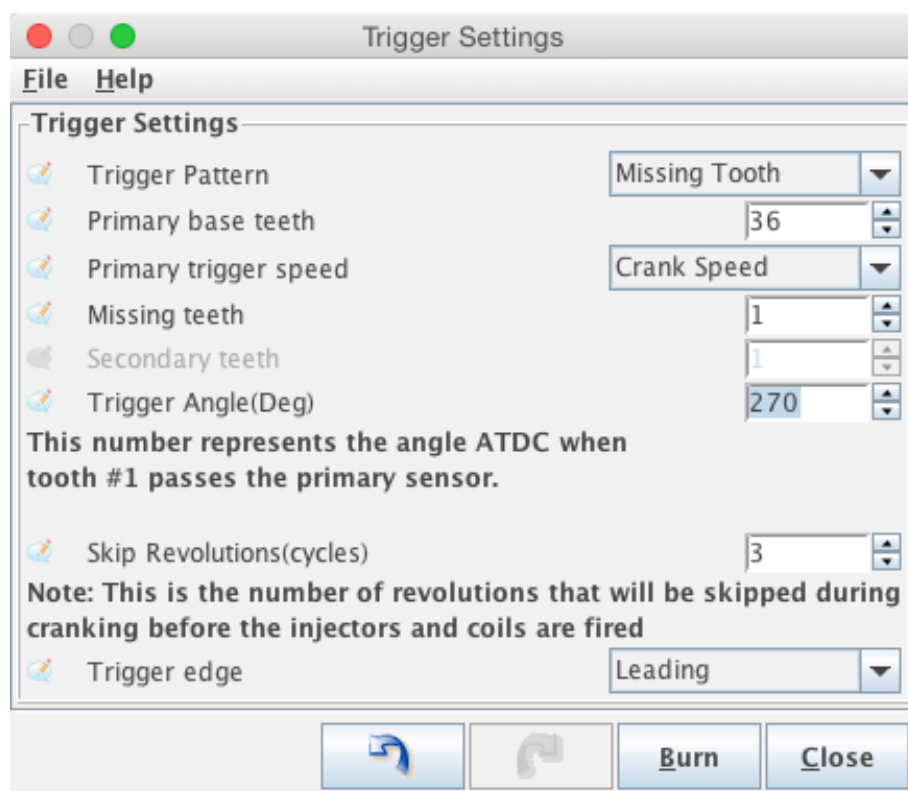
It is comprised of crank wheel with a given number of evenly spaced teeth, and one or more 'missing' teeth. Common values are typically 60-2, 36-1, 24-1, 12-1 and 4-1 where the first number represents the total number of teeth the wheel would have if there were none missing. The second number after a dash "-" indicates the number of teeth missing.

Note: If there is a third number (e.g., 36-1-1), the missing teeth are not sequential, and this decoder does not apply. Do not confuse counts with slashes "/", as numbers following slashes represent cam teeth—not missing teeth. Wheels with "+" indicate added teeth rather than missing, and again this decoder does not apply.

3.1.2 Applications

Missing tooth crank wheels can be used on virtually any engine and is one of the more popular aftermarket options. It provides very good resolution in the higher tooth count versions (Eg 36-1 and 60-2) without being CPU intensive to decode.

3.1.3 Tuner Studio Configuration



Fields:

- Primary base teeth: This is the number of teeth the wheel would have if there were none missing. Eg a 36-1 wheel has only 35 actual teeth, but you would enter 36 into this field.
- Missing Teeth: The size of the 'gap' in the number of teeth. These missing teeth must be situated in a single block (ie there's only a single gap in the teeth)
- Trigger Angle: This is the angle in crank degrees **AFTER** TDC (ATDC) of the first tooth following the gap

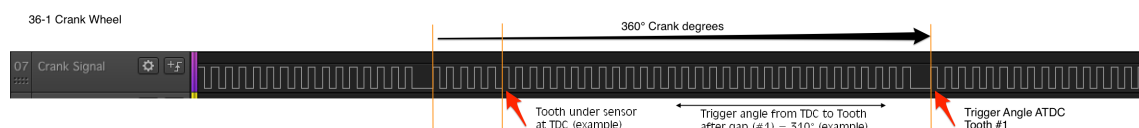
Timing Setting

Sequential operation

The missing tooth decoder supports sequential operation if an additional cam input is present. If Sequential mode is selected for either the fuel timing or spark timing, the system will expect to see a cam signal and will not sync correctly without this. Note that this is **ONLY** the case if sequential is selected for one or both of fuel and spark timing.

This cam signal should take the form of a single pulse every complete cycle. This can be a short tooth or a half moon type arrangement, provided that electrically there is only a single rising (or falling) pulse per cycle.

3.1.4 Trigger Pattern



3.2 Missing Tooth (cam)

3.2.1 Overview

The missing tooth cam-speed trigger is a Speeduino innovation, that permits function similar to a dual-wheel setup, thereby allowing sequential or wasted spark operation from cam-mounted or distributor wheels. The operation is based on both **Missing Tooth** and **Dual Wheel**. It is suggested to read those sections first for familiarization as this section will only highlight the fundamental differences to those common decoders.

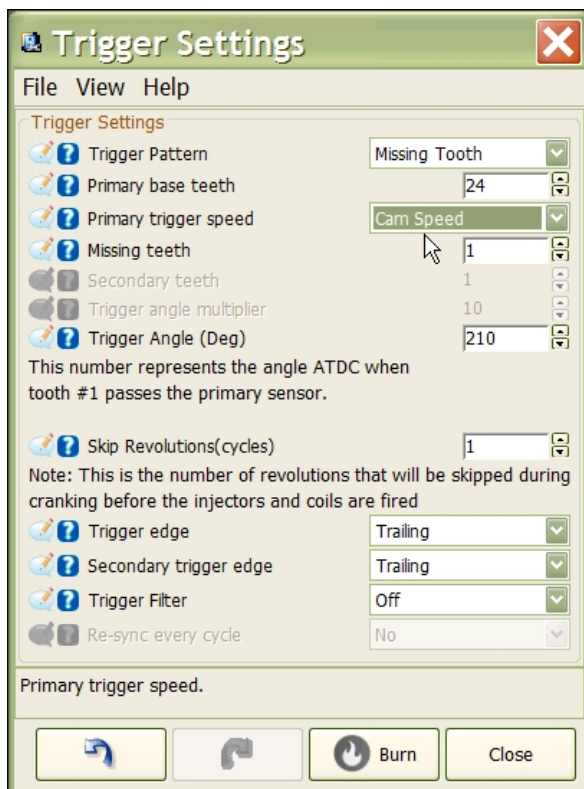
This decoder is comprised of a single cam-speed wheel in the same configuration as a crank-mounted missing-tooth wheel. The number of teeth **must** be evenly divisible into 720° . As it rotates at half crank speed, the sensor reads half the wheel teeth on each 360° crank revolution, and the remaining teeth on the next crank rotation. A single missing tooth will appear on only one of the two crank rotations, and is then used as a phase indicator, much as the dual-wheel system uses the cam signal.

3.2.2 Applications

Missing tooth cam or distributor wheels can be used with cam or distributor wheel modification or fabrication as no OEM systems use it originally. The wheel **must** have at least as many teeth as cylinders, **not** including the missing tooth. This generally requires double the number of teeth as cylinders or more. As many teeth, slots, or other readable features (sensor targets) as possible in the limited space is recommended in order to satisfy this requirement, and to maximize resolution. The sensor must be capable of reliably reading smaller or closely-spaced teeth.

Due to typically limited teeth, only half the teeth being read on each revolution, and the potential for reduced accuracy due to timing drive wear; the timing accuracy may be reduced in comparison to crank wheel systems. A figure of error cannot be predicted here as the wear or 'slop' of a given engine will be unique. However, it should be reasonable to assume the timing error will not exceed the accuracy of an OEM-equivalent cam-driven system such as typical distributor systems, or possibly better due to more sensor targets.

3.2.3 Tuner Studio Configuration

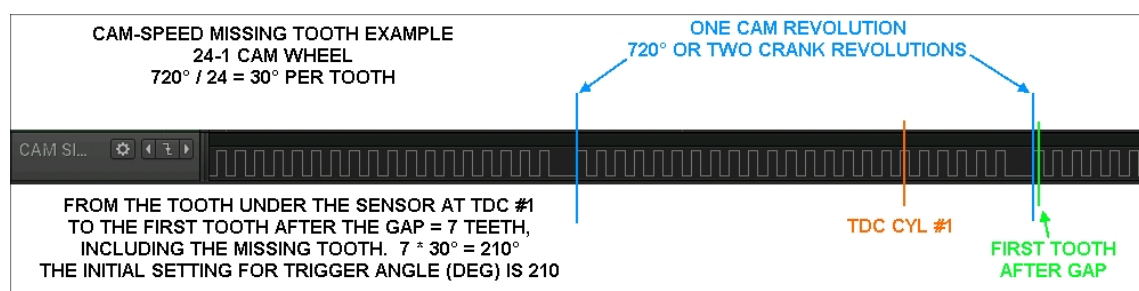


Fields:

- **Primary base teeth:** This is the number of teeth the wheel would have if there were none missing, e.g. a 36-1 wheel has only 35 actual teeth, but you would enter 36 into this field.
- **Missing Teeth:** The size of the 'gap' in the number of teeth. These missing teeth must be situated in a single block (ie there's only a single gap in the teeth). One missing tooth is recommended.
- **Trigger Angle:** This is the angle in crank degrees **AFTER** TDC (ATDC) of the first tooth following the gap. This number may exceed 360°.
- **Cam Speed:** Ensure this box is checked for this cam-speed system.

Timing Setting

3.2.4 Trigger Pattern



3.3 Dual Wheel

3.3.1 Overview

This decoder is used when there are two wheels. The primary signal must be based on crank speed and not from a missing tooth wheel, the secondary can be either from a crank or a cam wheel with only one tooth ('Half moon' is fine). Tooth #1 is defined to be the first tooth on the primary wheel **AFTER** the pulse on the secondary wheel.

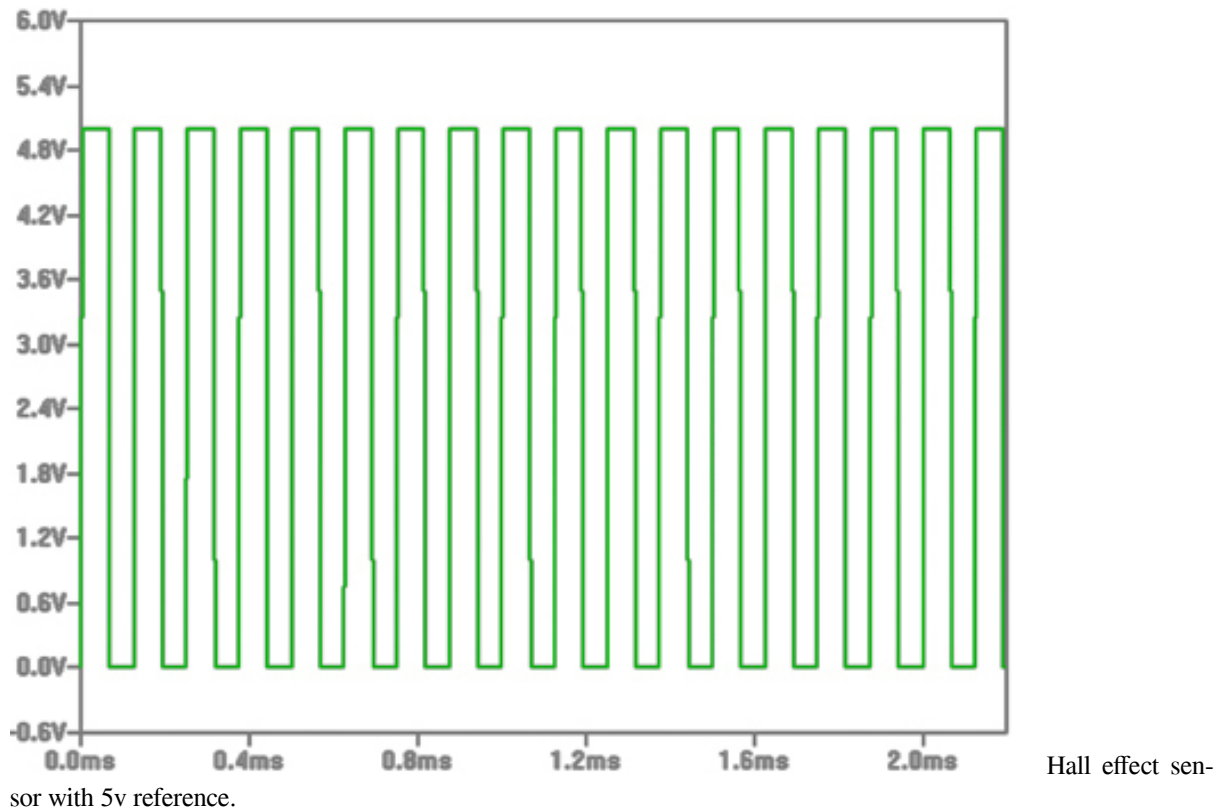
If the primary wheel is actually on the cam, simply divide it's teeth by two in Tuner Studio's trigger configuration to get crank speed.

3.4 Basic Distributor

3.4.1 Overview

This is a signal that lacks the cylinder position signal. Without a missing/added tooth or camshaft signal reference, Speeduino cannot calculate crankshaft angle, phase of cycle, or cylinder assignment. A distributor must be used to route the resulting sparks to the correct cylinders. The signal can be as simple as the breaker points from an old pre-electronic distributor, to a crankshaft wheel without any abnormal, extra, or missing slots. Most who have installed aftermarket tachometers are familiar with the simplicity of the signal with the only variation being the number of pulses in each crankshaft rotation.

3.4.2 Trigger Signal



3.5 GM 7X

3.5.1 Overview

This decoder uses a GM trigger wheel with six notches spaced evenly apart and one uneven notch. The uneven notch is counted as #3 with a total of seven notches.

3.6 4G63

3.6.1 Overview

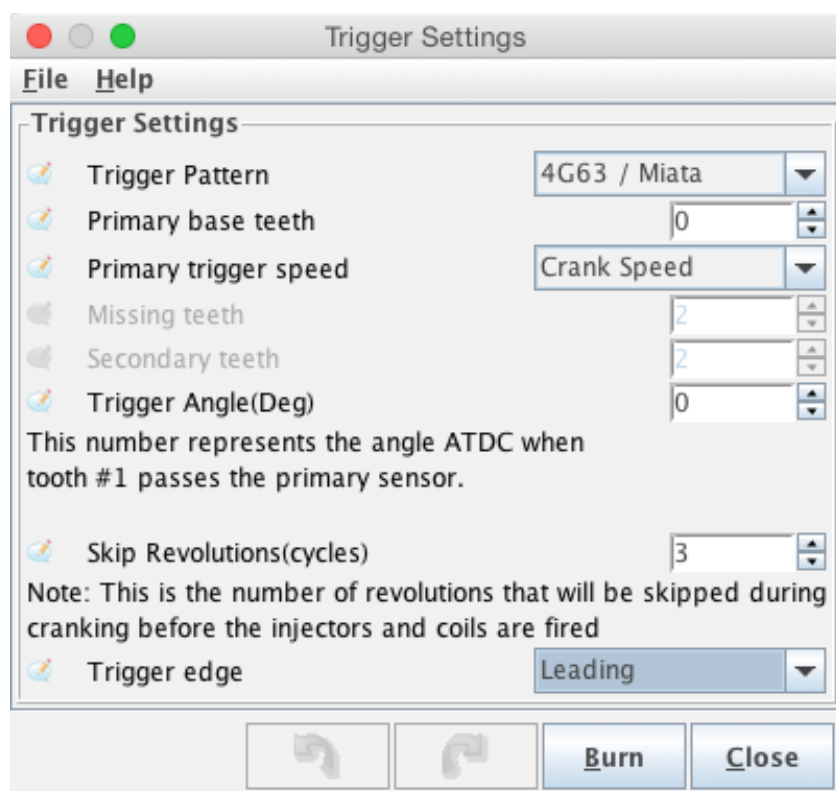
The 4g63 trigger is used across a large number of both Mitsubishi and Mazda 4 cylinder engines. See below for applications.

It is comprised of crank and cam signals that are provided by either a hall sensor or an optical sensor. The signal is electrically the same in both cases.

3.6.2 Applications

- Mitsubishi Lancer
- NA and NB Miata / MX-5

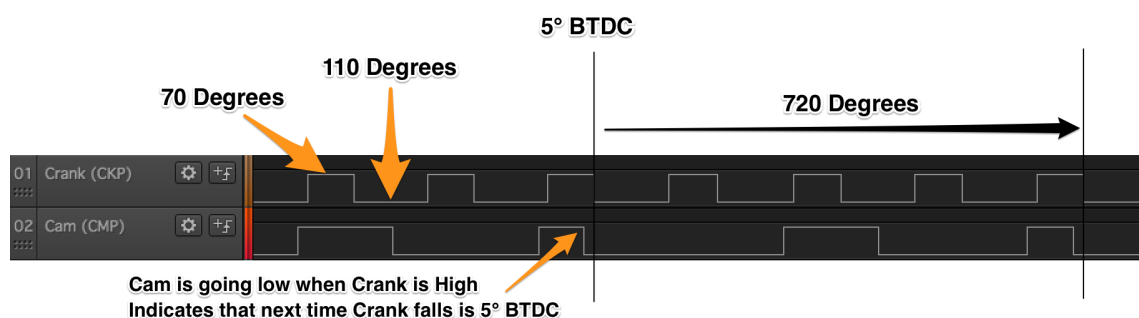
3.6.3 Tuner Studio Configuration



3.6.4 Timing adjustment

In most cases altering the trigger angle should not be required, however there is some small variation between the OEM versions of this trigger so some minor adjustment may be needed. Once you have the engine started, set a fixed ignition angle and check the timing with a timing light. If this is a few degrees out ($<20^\circ$), adjust the trigger angle here. If this is more than 20° out, there may be a larger problem.

3.6.5 Trigger Pattern

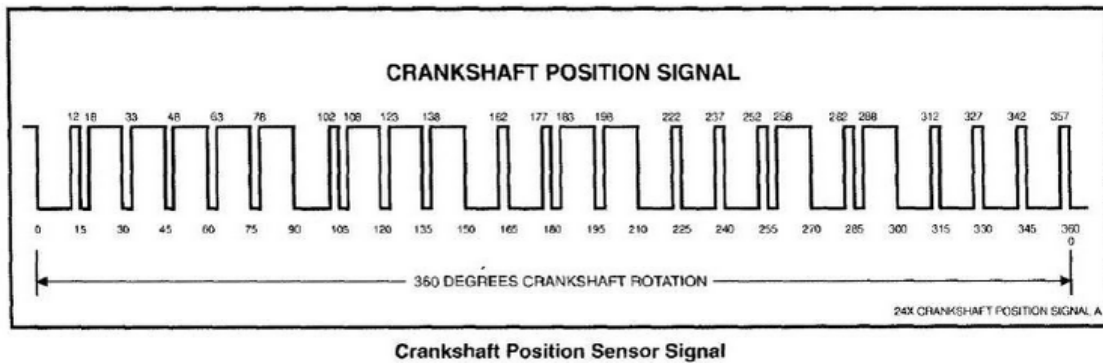


3.7 GM 24X

3.7.1 Overview

This is a 24 tooth wheel with 12 wide teeth and 12 narrow teeth. The narrow provides 3 degrees of pulse while the wide provides 12. All of the falling edges are 15 degrees apart. This decoder uses the falling edges, requiring the cam signal to determine crank angle.

3.7.2 Trigger Signal

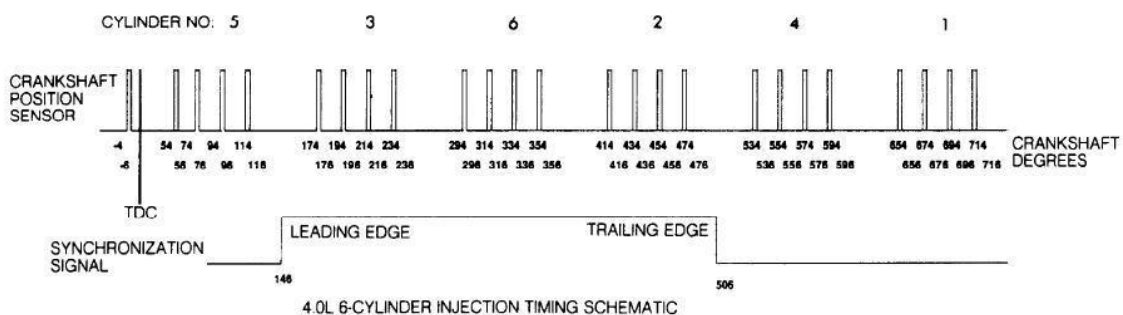


3.8 Jeep 2000

3.8.1 Overview

There are two signals one from the crank wheel and the other from the cam. The crank wheel puts out a series of four pulses every 120 degrees. Each of the four pulses is 20 degrees apart and lasting only 2 degrees. The cam wheel pulses once every 360 degrees or 720 crank degrees. The pulse last for 180 degrees or 360 crank degrees.

3.8.2 Trigger Signal



3.9 Audi 135

The Audi 135 decoder is for engines using 135 pulses-per-revolution, such as Audi V6, I5, I4T, etc. It was developed prior to the Non-360 "uneven divisor" decoder, but is similar in basis.

3.10 Honda D17

3.10.1 Overview

The Honda D17 decoder applies to the Honda engine family using a 12+1 crankshaft wheel. The 4+1 camshaft signal is not currently used with Speeduino. Without the cam signal, all standard fuel and ignition modes up to semi-sequential and wasted-spark are supported.

3.10.2 Applications

- TBA

3.10.3 Tuner Studio Configuration

3.10.4 Timing adjustment

In most cases altering the trigger angle should not be required, however there is some small variation between the OEM versions of this trigger so some minor adjustment may be needed. Once you have the engine started, set a fixed ignition angle and check the timing with a timing light. If this is a few degrees out ($<20^\circ$), adjust the trigger angle here. If this is more than 20° out, there may be a larger problem.

3.10.5 Trigger Pattern

The crank trigger wheel consists of 12 evenly spaced teeth plus 1 additional 13th tooth which provides position information. The first tooth after this 13th one is considered Tooth #1



3.11 Miata 99

EXPERIMENTAL - intended for Mazda Miata 1999-2000. Refer to the [Speeduino Forums](#) for more info.

3.12 Non-360

3.12.1 Non-360 Decoder

This is a variation of the dual-wheel decoder that can be used with tooth counts that do not divide evenly into 360° . This decoder system is usually unique to a particular brand or engine series, and therefore has previously been assigned a name to identify the type, such as the Audi 135 decoder. While this “uneven divisor” decoder can be used with a variety of tooth counts, not all tooth counts can be used with this system.

3.13 Nissan 360

3.13.1 Overview

The Nissan 360 CAS trigger is used across a large number of both 4 and 6 cylinder Nissan engines. See below for applications.

The trigger is comprised of a wheel, running at cam speed, that has 360 windows and is read by an optical sensor. Each window therefore represents 2 crank degrees. For location information, there is also an inner ring of windows, equal to the number of cylinders (ie 4 windows on 4 cylinder engines, 6 windows on 6 cylinder engines).

NOTE: There are multiple versions of the 4 cylinder CAS and not all are currently supported. Each known version is described below

1. Pattern 1 - Has a single unique inner window with all others being identical. Not currently supported
2. Pattern 2 - The unique slot sizes are in opposing pairs. This is partially supported.
3. Pattern 3 - Each inner window has a unique size. Typically 4,8,12,16 on 4 cylinder engines and 4,8,12,16,20,24 on 6 cylinders. This is supported.

3.13.2 Applications

- CA18 - Believed to have pattern 2
- SRxx Redtop - Unknown
- SRxx Blacktop (early) - Believed to be pattern 3
- SRxx Blacktop (notch) - Believed to be pattern 3
- FJ20 - Believed to have pattern 1
- RB30 - Believed to have pattern 1
- RB25/26 - Believed to all have pattern 3

3.13.3 Tuner Studio Configuration

TBA

3.13.4 Timing adjustment

In most cases altering the trigger angle should not be required, however there is some small variation between the OEM versions of this trigger so some minor adjustment may be needed. Once you have the engine started, set a fixed ignition angle and check the timing with a timing light. If this is a few degrees out (<20°), adjust the trigger angle here. If this is more than 20° out, there may be a larger problem.

3.13.5 Trigger Pattern

TBA

3.14 Subaru 6/7

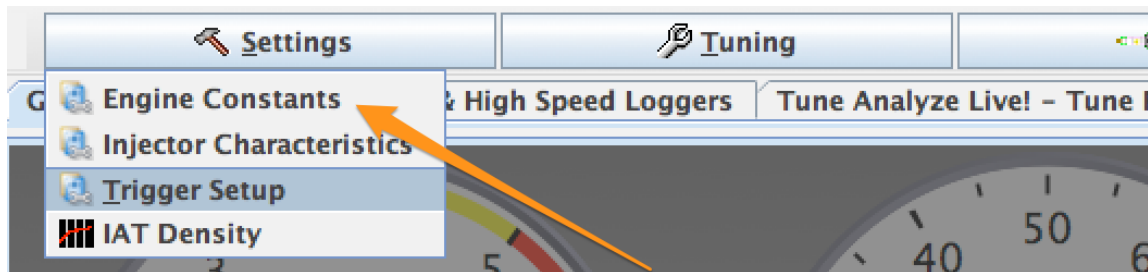
This article is a stub. You can help Speeduino by [expanding it](#).

Chapter 4

Configuration

4.1 Engine Constants

From the Settings menu, select Constants



Here you need to setup the engine constants. Fill out the fields in the bottom section before calculating the Required Fuel.

The **ONLY** currently supported combinations for Squirts Per Engine Cycle and 'Injector Staging' are:

All other combinations will currently cause issues (some small, some large).

For sequential installations, use the 'Alternating' setting with 2 squirts (Internally Speeduino adjusts this to 1)

4.1.1 Settings[edit]

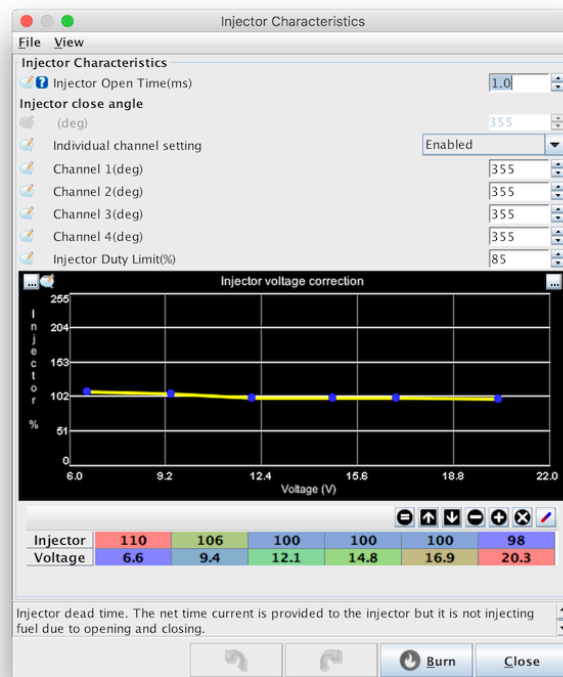
- **Injector Staging:** This configures the timing strategy used for the injectors
 - Alternating (Recommended for most installs) - Injectors are timed around each cylinders TDC. The exact closing angle can be specific in the Injector Characteristics dialog.
 - Simultaneous - All injectors are fired together, based on the TDC of cylinder 1.
- **Engine stroke:** Whether the engine is 2 stroke or 4 stroke
- **Number of cylinders:** Number of cylinders in the engine
- **Injector Port Type:** Option isn't used by firmware. Selection currently does not matter
- **Number of injectors:** Usually the same as number of cylinders (For port injection)
- **Engine Type:** Whether the crank angle between firings is the same for all cylinders. If using an Odd fire engine (Eg Some V-Twins and Buick V6s), the angle for each output channel must be specific.
- **Injector Layout:** Specifies how the injectors are wired in
 - **Paired:** 2 injectors are wires to each channel. The number of channels used is therefore equal to half the number of cylinders.
 - **Semi-Sequential:** Semi-sequential: Same as paired except that injector channels are mirrored (1&4, 2&3) meaning the number of outputs used are equal to the number of cylinders. Only valid for 4 cylinders or less.
 - **Sequential:** 1 injector per output and outputs used equals the number of cylinders. Injection is timed over full cycle. Only available for engines with 4 or fewer cylinders.
- **Board Layout:** Specifies the input/output pin layout based on which Speeduino board you're using. For specific details of these pin mappings, see the utils.ino file
- **MAP Sample Method:** How the MAP sensor readings will be processed:
 - **Instantaneous:** Every reading is used as it is taken. Makes for a highly fluctuating signal, but can be useful for testing

- **Cycle Average:** The average sensor reading across 720 crank degrees is used. This is the recommended option for 4 or more cylinders
- **Cycle Minimum:** The lowest value detected across 720 degrees is used. This is the recommended method for less than 4 cylinders or ITBs

4.2 Injector Characteristics

4.2.1 Overview

4.2.2 Settings



4.3 Trigger Setup

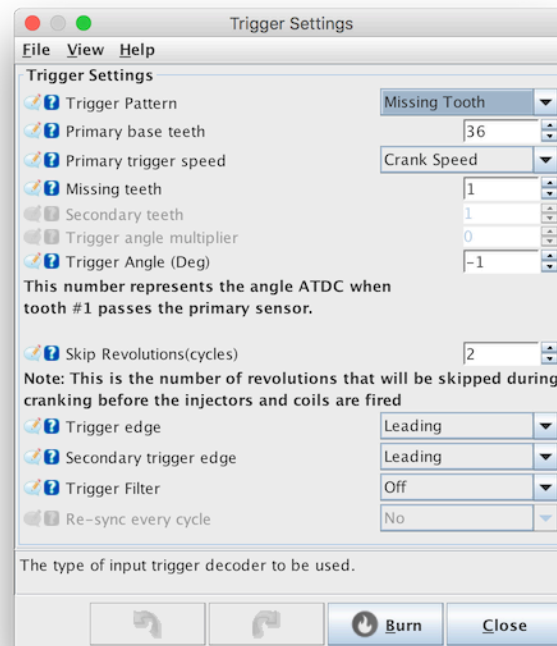
4.3.1 Overview

One of the most critical components of an EFI setup is the Crank Angle Sensor (CAS) and how it is used by the ECU. The Trigger settings dialog is where the trigger configuration is defined and it is vitally important to have this correct before trying to start your engine.

With incorrect settings, you may have issues getting sync or see erratic RPM readings.

Note that many of the settings on this dialog are dependant on your configuration and it is therefore normal that some options maybe greyed out.

4.3.2 Trigger Settings



- **Trigger Pattern** - The pattern used by the crank/cam sensor setup on your engine. For a full list of the supported patterns, see: <https://speeduino.com/wiki/index.php/Decoders>
- **Primary Base teeth** - For patterns where the number of teeth are variable (missing tooth, dual wheel etc), this number represents the number of teeth on the primary wheel. For missing tooth type wheels, this number should be the count as if there were no teeth missing.
- **Primary trigger speed** - The speed at which the primary input spins. It is closely related to the Primary Base teeth setting and indicates whether that number of teeth passes the sensor once every crank revolution or every cam revolution.
- **Missing teeth** - If using the missing tooth pattern, this is the size of the gap, given in 'missing teeth'. Eg 36-1 has 1 missing tooth. 60-2 has 2 missing teeth etc. The missing teeth **MUST** be all located in a single block, there cannot be multiple missing tooth gaps around the wheel.
- **Secondary teeth** - As above, but for the secondary input. This input is always assumed to run at cam speed.
- **Trigger angle multiplier** -
- **Trigger angle** - The angle of the crank, **After Top Dead Centre (ATDC)**, when tooth #1 passes the sensor on the primary (crank) input. This setting is critical for Speeduino to accurately know the current crank angle. See section below ('Finding tooth #1 and trigger angle') for further information on how to determine this value

Trigger options

- **Skip revolutions** -
- **Trigger edge** -
- **Secondary trigger edge** -
- **Trigger filter** -
- **Re-sync every cycle** -

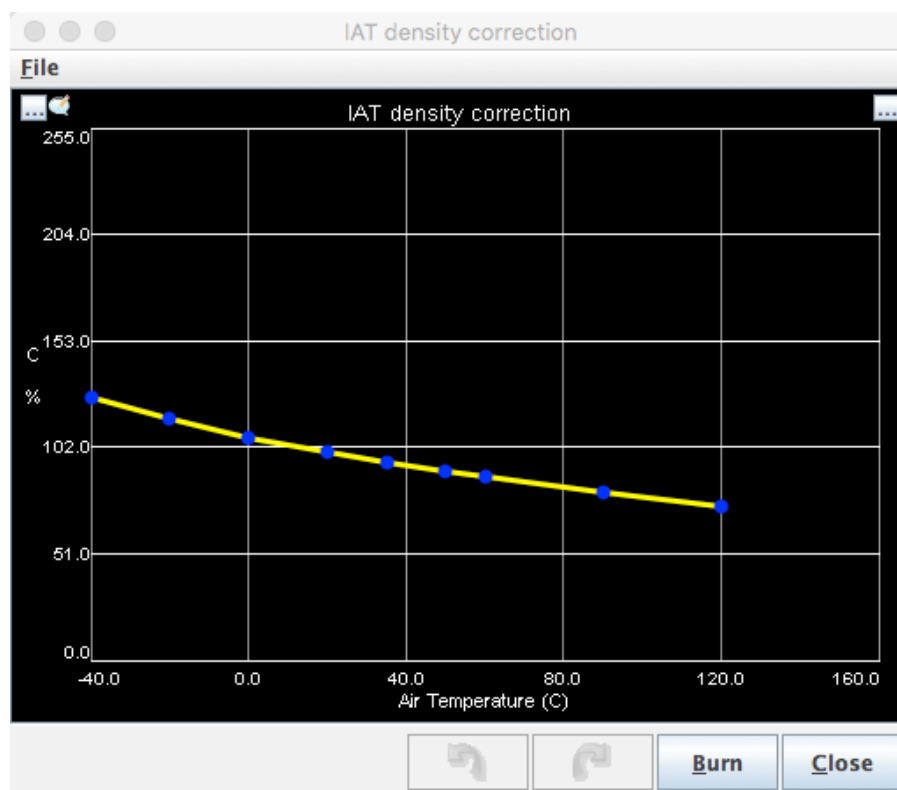
4.3.3 Finding tooth #1 and trigger angle

4.4 IAT Density

4.4.1 Overview

The IAT density curve represents the change in oxygen density of the inlet charge as temperature rises. The default curve approximately follows the ideal gas law and is suitable for most installations, however if you are seeing very high inlet temperatures (Either due to heat soak in the engine bay or from turbocharging) the you may need to adjust the hot end of this curve.

4.4.2 Settings



4.5 Acceleration Enrichment

4.5.1 Overview

Acceleration Enrichment (AE) is used to add extra fuel during the short transient period following a rapid increase in throttle. It performs much the same function as an accelerator pump on a carbureted engine, increasing the amount of fuel delivered until the manifold pressure reading adjusts based on the new load.

To operate correctly, you must have a variable TPS installed and calibrated.

4.5.2 Theory

Tuning of acceleration enrichment is based on the rate of change of the throttle position, a variable known as TPSdot (TPS delta over time). This is measured in %/second, with higher values representing faster presses of the throttle and values in the range 50%/s to 1000%/s are normal. Eg:

- 100%/s = pressing the throttle from 0% to 100% in 1 second
- 1000%/s = pressing the throttle from 0% to 100% in 0.1s

TPSdot forms the X axis of the acceleration curve, with the Y axis value representing the % increase in fuel.



Tuning

The enrichment curve included with the base Speeduino tune is a good starting point for most engines, but some adjustment is normal depending on injector size, throttle diameter etc.

In most cases, tuning of the AE curve can be performed in a stationary environment, though dyno or road tuning is also possible. Fast and slow blips of the throttle should be performed and the affect on the AFRs monitored using the live line graph on the AE dialog. This graph shows both TPSdot and AFR values in sync with each other, making adjustments to the correct part of the AE curve simpler to identify.

If you find that the AFR is initially good, but then goes briefly lean, you should increase the 'Accel Time' setting, with increments of 10-20ms recommended.

False triggering In cases where the TPS signal is noisy, spikes in its reading may incorrectly trigger the acceleration enrichment. This can be seen in a log file or on a live dash in TunerStudio by the activation of the 'TPS Accel' indicator when there is no (or little) throttle movement occurring. Should this occur (and assuming that the TPS wiring cannot be corrected to reduce noise) then the false triggers can be prevented from triggering AE by increasing the "TPSdot Threshold" value. This should be increased in increments of ~5%/s, pausing between each increase to observe whether AE is still being incorrectly activated.

4.6 Rev Limits

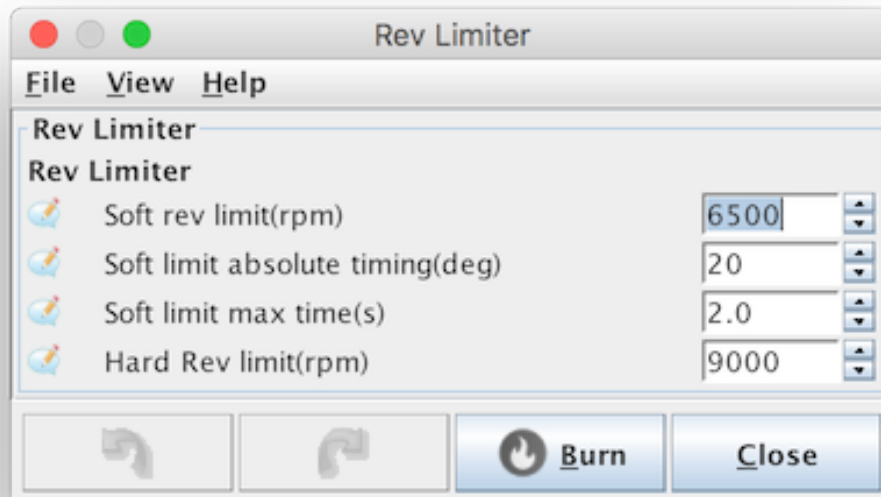
4.6.1 Overview

Speeduino includes a spark based rev limited with both hard and soft cuts.

The soft cut limiter will lock timing at an absolute value to slow further acceleration. If RPMs continue to climb and reach the hard cut limit, ignition events will cease until the RPM drop below this threshold.

Note As this is spark based limiting, fuel only installs cannot use the rev limiter functionality

4.6.2 Settings



- **Soft rev limit:** The RPM at which the soft cut ignition timing will be applied over.
- **Soft limit absolute timing:** Whilst the engine is over the soft limit RPM, the ignition advance will be held at this value. Lower values here will have a greater soft cut affect.
- **Soft limit max time:** The maximum number of seconds that the soft limiter will operate for. If the engine remains in the soft cut RPM region longer than this, the hard cut will be applied.
- **Hard rev limiter:** Above this RPM, all ignition events will cease.

4.7 Flex Fuel

4.7.1 Overview

Speeduino has the ability to modify fuel and ignition settings based on the ethanol content of the fuel being used, a practice typically known as flex fuelling. A flex fuel sensor is installed in the feed or return fuel lines and a signal wire is used as an input on the Speeduino board.

As ethanol is less energy dense, but also has a higher equivalent octane rating, adjustments to the fuel load and ignition timing are required.

4.7.2 Hardware

Speeduino uses any of the standard GM/Continental Flex fuel sensors that are widely available and were used across a wide range of vehicles. These were available in 3 different units, all of which are functionally identical, with the main difference being only the physical size and connector. The part numbers for these are:

- Small - #13577429
- Mid-size - #13577379
- Wide - #13577394 (Same as the mid-size one, but with longer pipes)

All 3 use a variant of the Delphi GT150 series connector. You can use a generic GT150 connector, but you will have to clip off 2 tabs from the side of the sensor.

Part numbers :

- Housing (#13519047): <http://au.mouser.com/ProductDetail/Delphi-Connection-Systems/13519047/?qs=%2Fha2pyFaduhVbz3D>
- Pins (#15326427): <http://au.mouser.com/ProductDetail/Delphi-Connection-Systems/15326427/?qs=sGAepiMZZMs7eK6h2E3D>
- Seal (#15366021): <http://au.mouser.com/ProductDetail/Delphi-Connection-Systems/15366021/?qs=sGAepiMZZMs7eK6h2E252b3Cc%3D>

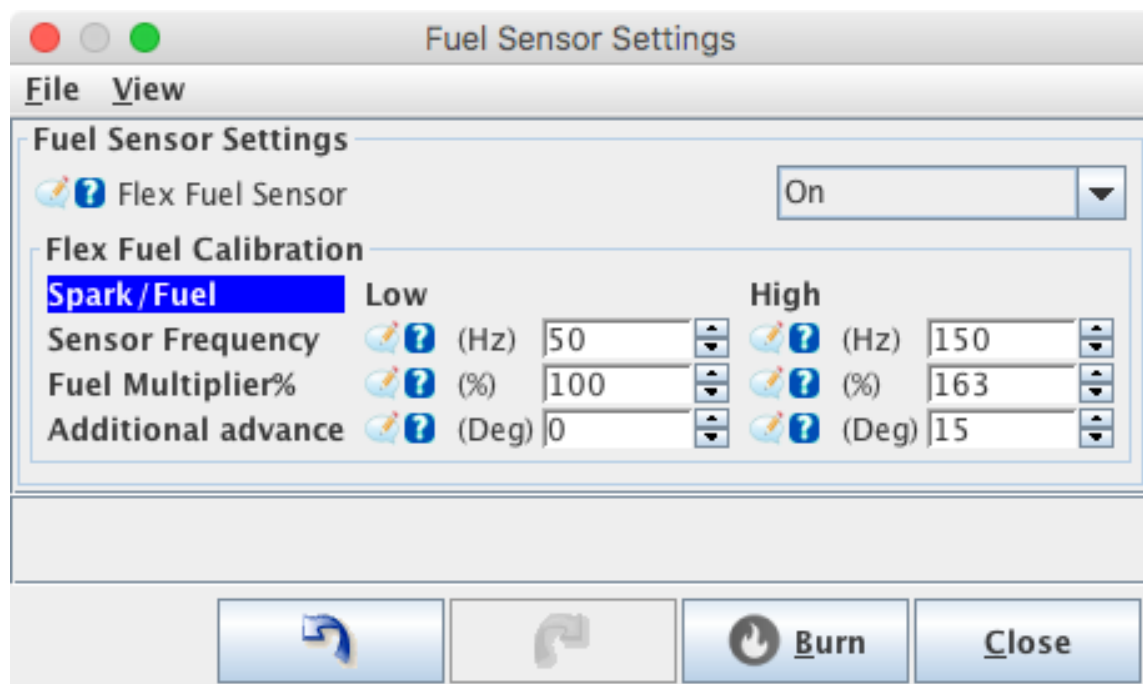
Alternatively, there is a GM part for a harness connector, part number 13352241: <http://www.gmpartsdirect.com/oe-gm/13352241>

Wiring

All units are wired identically and have markings on the housing indicating what each pin is for (12v, ground and signal) Speeduino boards v0.3.5+ and v0.4.3+ have an input location on their proto areas that the signal wire can be directly connected to.

On boards earlier to these, you will need to add a pullup resistor of between 2k and 3.5k Ohm. Recommended value is 3.3k, however any resistor in this range will work. Note that this is a relatively strict range, more generic values such as 1k or 10k DO NOT WORK with these sensors.

4.7.3 Tuning



- **Sensor frequency** - The minimum and maximum frequency of the sensor that represent 0% and 100% ethanol respectively. For standard GM/Continental flex sensors, these values are 50 and 150

- **Fuel multiplier%** - This is the additional fuel that should be added as ethanol content increases. The Low value on the left represents the adjustment to the fuel map at 0% ethanol and will typically be 100% if the base tune was performed with E0 fuel. If the base tune was made with E10 or E15 however, this value can be adjusted below 100%. The high value represents the fuel multiplier at 100% ethanol (E100) and the default value of 163% is based on the theoretical difference in energy density between E0 and E100. Tuning of this value may be required
- **Additional advance** - The additional degrees of advance that will be applied as ethanol content increases. This amount increases linearly between the low and high values and is added after all other ignition modifiers have been applied.

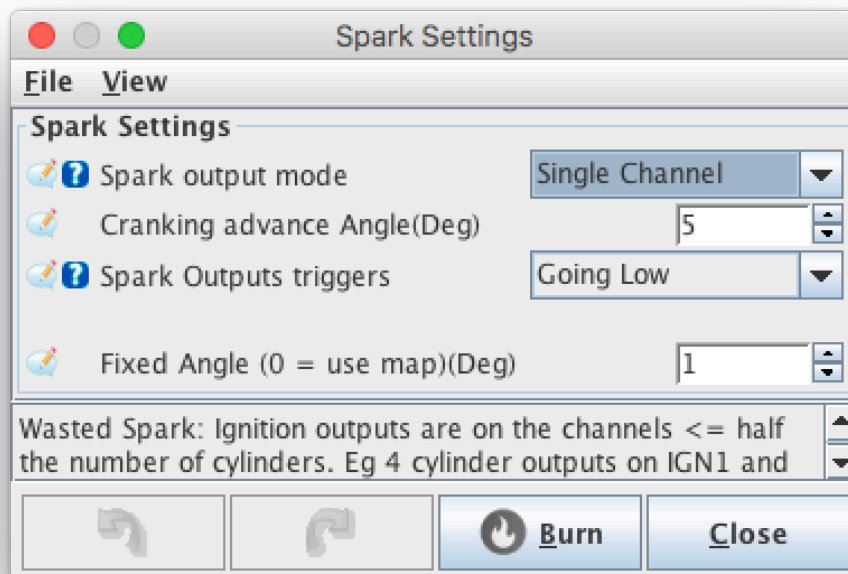
4.8 Spark Settings

4.8.1 Overview

The Spark settings dialog contains the options for how the ignition outputs will function, including which of the 4 IGN outputs are used and how. They are critical and incorrect values will result in an engine not starting and in some cases damage to hardware is possible. This dialog also contains a number of options for fixing the ignition timing for testing and diagnosis.

Please ensure you have reviewed these settings prior to attempting to start your engine.

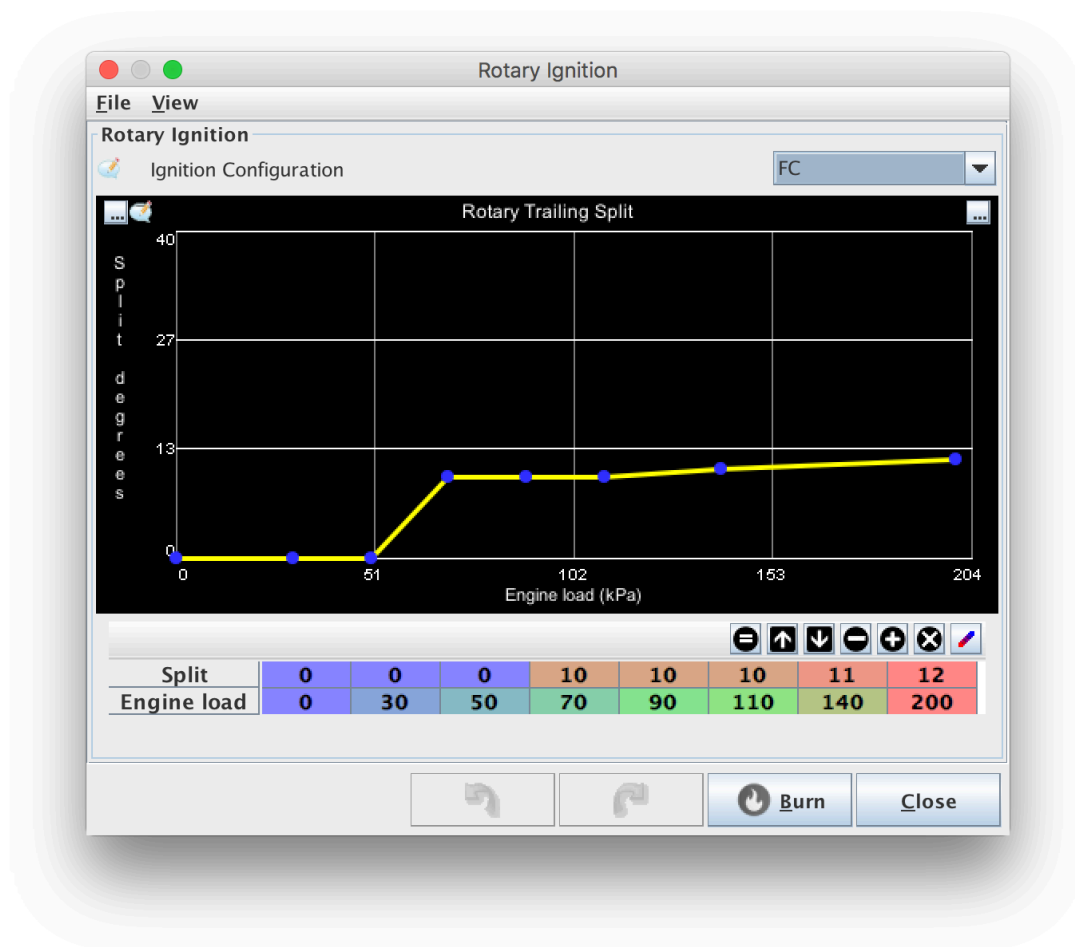
4.8.2 Settings



- **Spark Output mode** - Determines how the ignition pulses will be outputted and is very specific to your ignition wiring. **Note that no matter which option is selected here, ignition signals ALWAYS fire in numerical order (ie 1->2->3->4) up to the maximum number of outputs.** The firing order of the engine is accounted for in the wiring order.

- **Wasted Spark** - Number of ignition outputs is equal to have the number of cylinders and each output will fire once every crank revolution. One spark will therefore take place during the compression stroke and the other on the exhaust stroke (aka the 'wasted' spark). This method is common on many 80s and 90s vehicles that came with specific wasted spark coils, but can also be used with individual coils that are wired in pairs. Wasted spark will function with only a crank angle reference (Eg a missing tooth crank wheel with no cam signal)
- **Single Channel** - This mode sends all ignition pulses to IGN1 output and is used when the engine contains a distributor (Typically with a single coil). The number of output pulses per (crank) revolution is equal to half the number of cylinders.
- **Wasted COP** - This is a convenience mode that uses the same timing as the 'Wasted Spark' option, however each pulse is sent to 2 ignition outputs rather than one. These are paired IGN1/IGN3 and IGN2/IGN4 (ie When IGN1 is high, IGN3 will also be high). As this is still a wasted spark timing mode, only crank position is required and there will be 1 pulse per pair, per crank revolution. This mode can be useful in cases where there are 4 individual coils, but running full sequential is either not desired or not possible (Eg when no cam reference is available).
- **Sequential** - This mode is only functional on engines with 4 or fewer cylinders.
- **Rotary** - See below for full detail
- **Cranking advance** - The number of absolute degrees (BTDC) that the timing will be set to when cranking. This overrides all other timing advance modifiers during cranking.
- **Spark output triggers** - **THIS IS A CRITICAL SETTING!**. Selecting the incorrect option here can cause damage to your igniters or coils. Specifies whether the coil will fire when the ignition output from Speeduino goes HIGH or goes LOW. The VAST majority of ignition setups will require this to be set **GOING LOW** (ie the coil charges/dwells when the signal is high and will **fire** when that signal goes low). Whilst GOING LOW is required for most ignition setups, there are some configurations that perform the dwell timing on the ignition module and fire the coil only when they receive a HIGH signal from the ECU.
- **Fixed Angle** - This is used to lock the ignition timing to a specific angle for testing. Setting this to any value other than 0 will result in that exact angle being used (ie overriding any other settings) at all RPMs/load points, except during cranking (Cranking always uses the above Cranking Advance setting). This setting should be set to 0 for normal operation.

Rotary modes



Speeduino currently only supports the ignition configuration used on FC RX7 engines. Support for FD and RX8 ignition setups is in development. The leading / trailing split angle can be set as a function of the current engine load.

- **FC** - Outputs are configured for the Leading/Trailing setup that was used on FC RX7s. Wiring required is:
 - **IGN1** - Leading spark
 - **IGN2** - Trailing spark
 - **IGN3** - Trailing select
- **FD** - Not currently supported
- **RX8** - Not currently supported

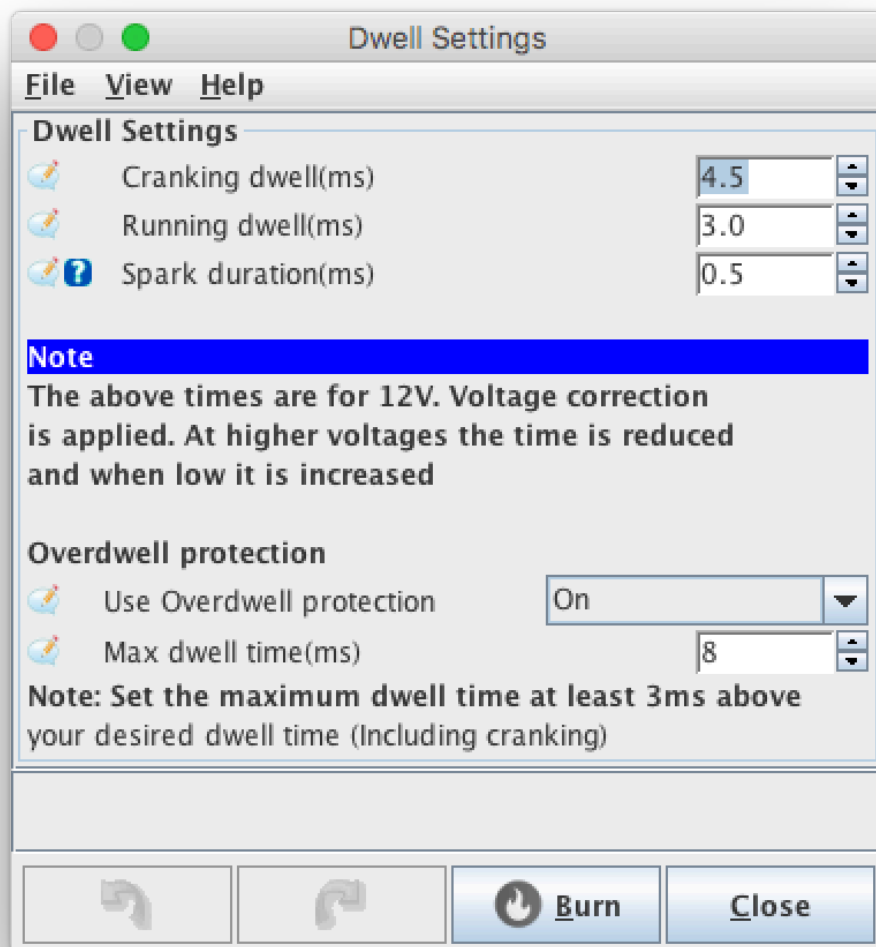
4.9 Dwell

4.10 Overview

The dwell control dialog alters the coil charging time (dwell) for Speeduino's ignition outputs. Care should be taken with these settings as igniters and coils can be permanently damaged if dwelled for excessive periods of time.

From the April 2017 firmware onwards, dwell will automatically reduce when the configured duration is longer than the available time at the current RPM. This is common in single channel ignition configurations (Eg 1 coil with a distributor) and in particular on higher cylinder count engines.

4.11 Settings



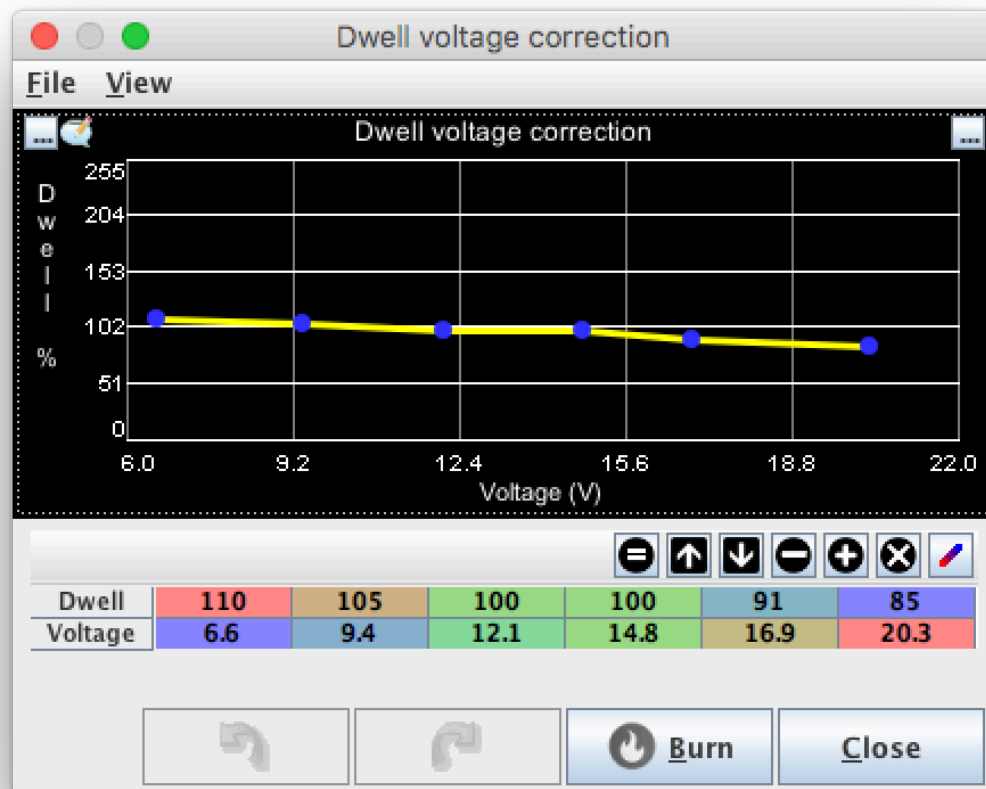
Note: Both the running and cranking dwell times are nominal values, assumed to be at a constant voltage (Usually 12v). Actual dwell time used will depend on the current system voltage with higher voltages having lower dwell times and vice versa. See section below on voltage correction

- Cranking dwell - The nominal dwell time that will be used during cranking. Cranking is defined as being whenever the RPM is above 0, but below the 'Cranking RPM' values in the **Cranking** dialog
- Running dwell - The nominal dwell that will be used when the engine is running normally.
- Spark duration - The approximate time the coil takes to fully discharge. This time is used in calculating a reduced dwell when in time limited conditions, such as mentioned above on single coil, high cylinder count engines. The limited dwell time is calculated by taking the maximum revolution time at the given RPM, dividing by the number of spark outputs required per revolution and subtracting the spark duration. Outside of those conditions, this setting is not used.
- Over dwell protection - The over dwell protection system runs independently of the standard ignition schedules and monitors the time that each ignition output has been active. If the active time exceeds this amount, the output will be ended to prevent damage to coils. This value should typically be at least 3ms higher than the nominal dwell times configured above in order to allow overhead for voltage correction.

4.11.1 Voltage correction

As the system voltage rises and falls, the dwell time needs to reduce and increase respectively. This allows for a consistent spark strength without damaging the coil/s during high system voltage conditions. It is recommended that 12v be used as the 'nominal' voltage, meaning that the Dwell % figure at 12v should be 100%.

The correction curve in the base tune file should be suitable for most coils / igniters, but can be altered if required.

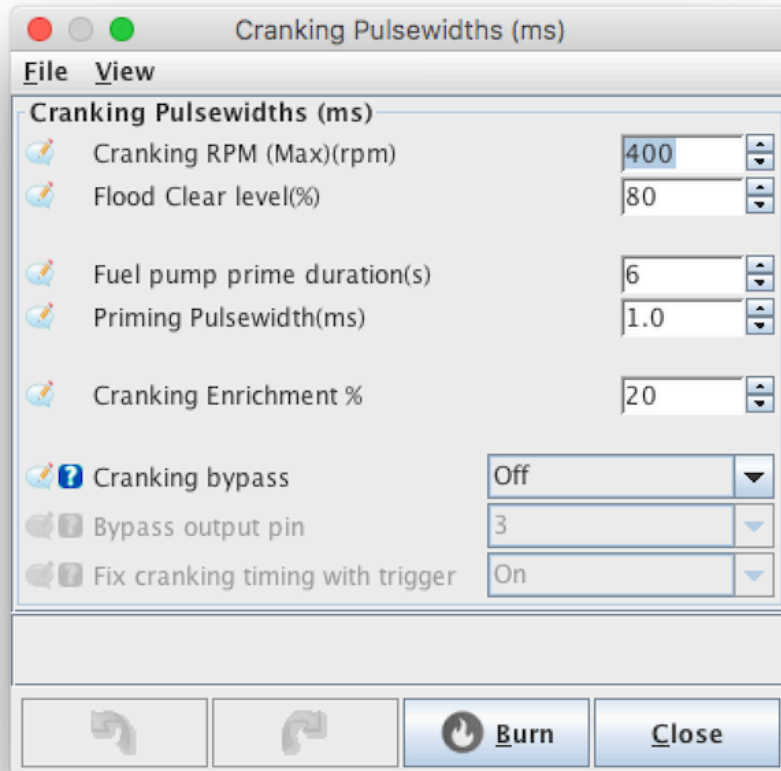


4.12 Cranking

4.12.1 Overview

Cranking conditions during starting typically require multiple adjustments to both fuel and ignition control in order to provide smooth and fast starts. The settings on this dialog dictate when Speeduino will consider the engine to be in a cranking/starting condition and what adjustments should be applied during this time.

4.12.2 Settings

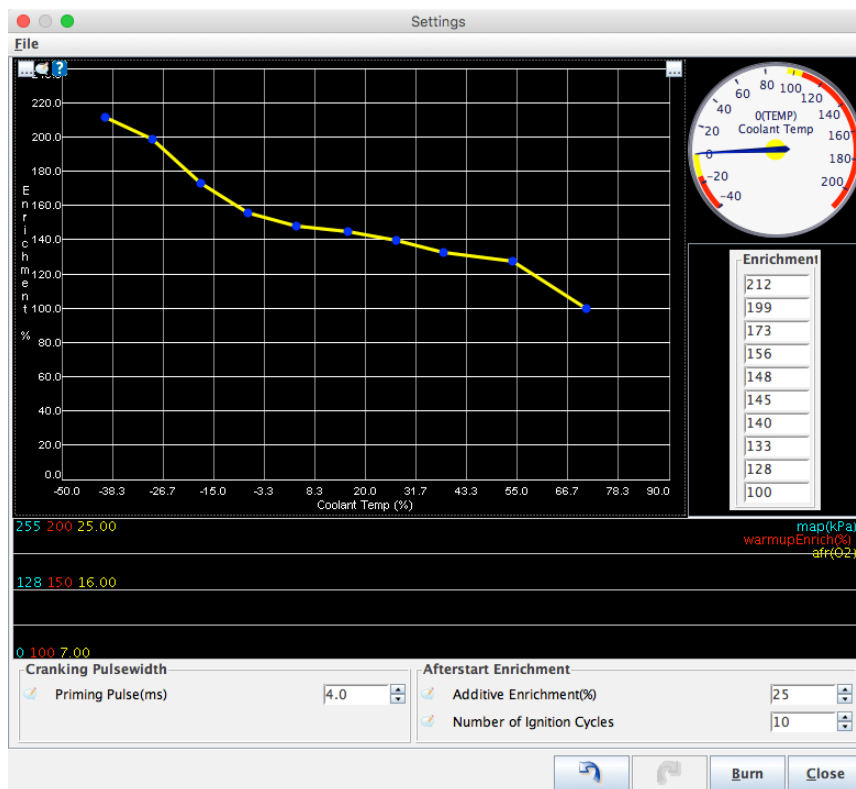


- **Cranking RPM** - This sets the threshold for whether Speeduino will set its status to be cranking or running. Any RPM above 0 and below this value will be considered cranking and all cranking related adjustments will be applied. It's generally best to set this to be around 100rpm higher than your typical cranking speed to account for spikes and to provide a smoother transition to normal idle
- **Flood Clear level** - Flood clear is used to assist in removing excess fuel that has entered the cylinder/s. Whilst flood clear is active, all fuel and ignition events will be stopped and the engine can be cranked for a few seconds without risk of starting or further flooding. To trigger flood clear, the RPM must be **below** the above Cranking RPM setting and the TPS must be **above** the threshold of this setting.
- **Fuel pump prime duration** - When Speeduino is first powered on, the fuel pump output will be engaged for this many seconds in order to pressurise the fuel system. If the engine is started in this time, the pump will simply keep running, otherwise it will be turned off after this period of time. Note that fuel pump priming only occurs at system power on time. If you have USB connected, Speeduino remains powered on even without a 12v signal.
- **Priming Pulsewidth** - Upon power up, Speeduino will fire all injectors for this period of time. This pulse is NOT intended as a starting fuel load, but is instead for clearing out air that may have entered the fuel lines. It should be kept short to avoid engine flooding.
- **Cranking enrichment** - Whilst cranking is active (See Cranking RPM above), the fuel load will be increased by this amount. Note that as a standard correction value, this cranking enrichment **is in addition** to any other adjustments that are currently active. This includes the warmup enrichment etc.
- **Cranking Bypass** - This option is specifically for ignition systems that have a hardware cranking ignition option. These systems were used throughout the 80s and early 90s and allowed ignition timing to be fixed and

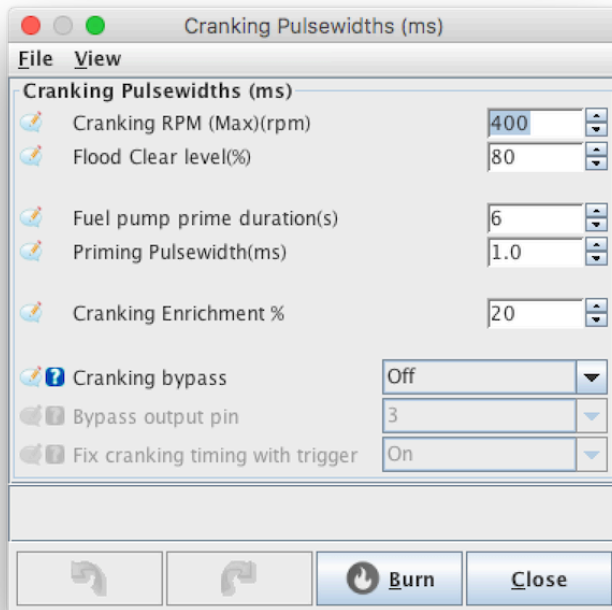
controlled by the ignition system itself when active (Via an input wire). With this option you can specify an output pin that will be set HIGH when the system is cranking. The pin number specified is the ARDUINO pin number.

- **Fix cranking timing with trigger** - Some (usually low resolution) trigger patterns are designed to align one of their pulses with the desired cranking advance. This is typically 5 or 10 degrees BTDC. When enabled, Speeduino will wait for this timed input pulse before firing the relevant ignition output (A dwell safety factor is still applied incase this pulse is not detected). This option is only made available when a trigger pattern that supports this function is selected (See [Trigger Setup](#))

4.13 Warmup



Cranking settings



4.14 Idle

4.14.1 Overview

Compatible Idle Valve Types

There are currently 3 modes of idle control available, using on/off, PWM duty cycle, or a stepper step count, enabled below a set coolant temperature. These modes cover the most common types of idle mechanisms in use. At this time only open loop control is available, meaning that an air bypass passage is enabled, rather than a target RPM. Closed loop control is anticipated at some point in the future, but no commitment is currently made.

Stand-Alone

While not an idle control mode, Speeduino is compatible with stand-alone idle valves that are self-controlling. Examples of this are thermal wax or bi-metal spring idle or auxiliary air valves like the one below. Internally expanding and contracting material opens and closes air valves, providing increased air flow and engine rpm when cold for warmup. Speeduino functions to enrich the cold engine and adjust for the additional air, in the same way it would if you opened the throttle slightly.



Other examples of stand-alone valves are simple On/Off valves as shown in the next section, controlled by inexpensive thermal switches like these:





On/Off

This is a simple digital on/off “switch” output by Speeduino that triggers at a selected temperature. It is intended to control an on/off fast idle valve as found in many older OEM setups, or an open/closed solenoid-type valve that is chosen for the purpose. In addition to OEM idle valves, examples of valves popular for re-purposing as on/off idle valves are larger vacuum, breather, or purge valves, and even fuel valves. Idle speed adjustment is generally set only once, with an in-line adjustable or fixed restrictor, pinch clamp, or other simple flow-control method.

Note: On/Off valves can be used in many ways to increase or decrease air flow for various idle purposes in-addition to warm-up. Examples are use as dashpot valves to reduce deceleration stalling, idle speed recovery for maintaining engine speed with accessory loads such as air conditioning, or air addition for specific purposes such as turbo anti-lag air control. See [Generic Outputs](#) for control information.

PWM

While similar in construction to many solenoid on/off valves; **PWM** idle valves are designed to vary the opening, and therefore flow through the valve, by PWM valve positioning.

Open-loop Duty Cycle Control Speeduino currently operates PWM valves in open-loop, effectively creating an on/off valve with adjustable flow. While the PWM duty cycle (DC) adjustment affects engine rpm, the adjustment is for valve opening, and therefore airflow and rpm are affected differently under various conditions. Note some idle valves default with no PWM signal to the open position, others closed, and some partially-open that close then re-open with increasing PWM DC. Be sure to research or test your valve type for proper operation.

PWM Settings Settings in TunerStudio include selecting PWM idle control, temperature and DC settings for warmup, and PWM DC during cranking under the following selections:

Under Idle control type, PWM is selected:

The temperature-versus-DC is selected under the Idle - PWM Duty Cycle selection. Note the relationship between temperature and PWM DC can be altered by simply moving the blue dots in the curve, or by selecting the table for manual entry as shown here:

Some engines prefer additional airflow during cranking for a reliable start. This air can be automatically added only during cranking by using the Idle - PWM Cranking Duty Cycle settings. Once the engine starts and rpm rise above the set maximum cranking rpm, the idle control switches to the previous warmup settings. Note the relationship between coolant temperature during cranking and PWM DC can be altered by simply moving the blue dots in the curve, or by selecting the table for manual entry as shown here:

NOTE: Every engine, valve type and tune is different. Suitable settings must be determined by the tuner. Do not infer any tuning settings from the images in this wiki. They are only random examples.

Dual idle output control is expected in the future, allowing 3-wire valves to be controlled more smoothly.

Stepper Motors

Stepper motor idle controls are very common on GM and other OEM setups. These motors typically have 4 wires (bi-polar). They must be driven through power transistors or a driver module, such as the **DRV8825 stepper motor driver** optional to the v0.4 board. These driver modules can be purchased inexpensively from a variety of vendors on sites such as eBay, Amazon, etc.

Most stepper idle valves function by turning a threaded rod in and out of the valve body in a series of partial-turn steps, increasing or decreasing airflow around the plunger (on end of valve below), and into the engine. The idle airflow bypasses the primary throttle body:

Example of a generic DRV8825 driver module on a v0.4 board:

Note the board is mounted at a standoff for air circulation and cooling:

The DRV8825 motor outputs are labeled as A2-A1-B1-B2, and the wiring connection examples are to this labeling. Check your schematics for the output connections that route to these DRV8825 outputs:

Examples of wiring to the DRV8825 driver:

The GM “screw-in” style used 1982 to 2003 on many models:

Stepper Driver Current Adjustment The DRV8825 stepper driver module includes a potentiometer (adjustable resistor) indicated by the yellow arrow in the image below. The potentiometer is used for setting the driver’s maximum current output limit. Because Speeduino uses full-step operation, the current limit is not critical to protect the module, but should be adjusted to the module’s maximum value for best operation of most automotive stepper IACs.

In order to set the potentiometer to maximum current before first use, ensure power to the module is OFF, then gently turn the potentiometer dial clockwise to the internal limit. **Do not force the adjustment beyond the internal stop.**

The module’s rated *continuous* current is up to 1.5A. While the module can supply a peak of 2.2A of current; in full-step mode and with the potentiometer adjusted to this position, the driver is limited to approximately 70% of full current, or approximately 1.5A.

Stepper Settings Settings in TunerStudio include selecting stepper idle control, temperature and step settings for warmup, and open steps during cranking under the following selections:

Under Idle control type, stepper is selected. The basic stepper operational settings are also located in this window:

Step time: This is how long (in ms) that the motor requires to complete each step. If this is set too low, the ECU will be trying to make the next step before the previous one is completed, which leads to the motor ‘twitching’ and not functioning correctly. If this is set longer than needed, the system will take longer to make each adjustment, which may lead to idle fluctuating more than desired. Typical values are usually 2ms - 4ms. The common GM stepper motor requires 3ms.

Home Steps: Stepper motors must be ‘homed’ before they can be used so the that ECU knows their current position. You should set this to the maximum number of steps that the motor can move.

Minimum steps: In order to allow a smooth idle that isn’t continually fluctuating, the ECU will only move the motor if at least this many steps are required. Typical values are in the 2-6 range, however if you have a noisy coolant signal line, this value may need to be increased.

The temperature-versus-steps is selected under the Idle - Stepper Motor selection. Note the relationship between temperature and motor steps can be altered by simply moving the blue dots in the curve, or by selecting the table for

manual entry as shown here:

Some engines prefer additional airflow during cranking for a reliable start. This air can be automatically added only during cranking by using the Idle - Stepper Motor Cranking settings. Once the engine starts and rpm rise above the set maximum idle rpm, the idle control switches to the previous warmup settings. Note the relationship between coolant temperature during cranking and motor steps can be altered by simply moving the blue dots in the curve, or by selecting the table for manual entry as shown here:

NOTE: Every engine, valve type and tune is different. Suitable settings must be determined by the tuner. Do not infer any tuning settings from the images in this wiki. They are only random examples.

NOTE: Refer to the *Pololu video* for instructions to set the DRV8825 driver current level to maximum for most automotive full-step stepper motors.



Examples **NOTE:** While normal DSM stepper function is seen at room temperatures at 3ms, step skipping occurs just under that speed. Very cold temperatures may cause skipping, thus the recommendation of 4ms. Test for the most suitable speeds for your setup.

Coll-attribution-page